



**Universidad
de Alcalá**

TRABAJO DE FIN DE GRADO

GRADO EN INGENIERÍA DE COMPUTADORES

ESCUELA POLITÉCNICA SUPERIOR

**Estimación de la intención de los usuarios de la
vía mediante redes neuronales basadas en
grafos**

Adrián Rodríguez Hurtado

Tutor: Augusto Luis Ballardini

Cotutor: Angie Nataly Melo Castillo

2025-2026



Escuela Politécnica Superior

GRADO EN INGENIERÍA DE COMPUTADORES

Trabajo Fin de Grado

Estimación de la intención de los usuarios de la vía mediante redes neuronales basadas en grafos

Autor: Adrián Rodríguez Hurtado

Tutor: Augusto Luis Ballardini

Cotutor: Angie Nataly Melo Castillo

TRIBUNAL:

Presidente: D. Miguel Ángel Sotelo Vázquez

Vocal 1º: D. Rubén Izquierdo Gonzalo

Vocal 2º: D. Iván García Daza

FECHA: 16/02/2026

Agradecimientos

Este trabajo ha sido posible gracias al apoyo incondicional de quienes me han acompañado y animado a lo largo del proyecto.

En primer lugar, quiero agradecer a mis padres y a mi hermana por toda la paciencia y apoyo que me han dado estos meses. Sin vuestro ánimo, nada de esto habría sido posible.

Al grupo de “Las Cabras”, por estar siempre ahí, confiando en que lo lograría.

A Víctor, Denys y Ximo, por todo el apoyo que me habéis dado durante este tiempo, estando pendientes del trabajo en todo momento.

Por último, a todas las personas que han estado implicadas en este proyecto, en especial a Augusto, que sin su ayuda tampoco hubiese sido posible el desarrollo de este trabajo.

Índice general

Índice general	I
Índice de figuras	IV
Índice de tablas	VI
Lista de acrónimos	VII
Resumen	VIII
Abstract	IX
1. Introducción	1
1.1. Motivación y contexto	1
1.2. Objetivos	2
1.3. Organización de la memoria	2
2. Estado del arte	4
2.1. Predicción de intención de peatones	4
2.2. Enfoques basados en grafos y Graph Neural Networks	4
2.3. Modelado temporal y multipeatón	5
3. Marco teórico	6
3.1. Inteligencia Artificial	6
3.2. Machine Learning	6
3.3. Grafos para el Machine Learning	7
3.4. Deep Learning	8
3.4.1. Redes neuronales	9
3.5. Redes neuronales basadas en grafos	10
3.6. Tipos de tareas en grafos	12

3.6.1. Node-level tasks	12
3.6.2. Edge-level tasks	12
3.6.3. Graph-level tasks	13
3.6.4. Resumen de tipos de tareas en grafos	13
3.7. Técnicas de embedding en grafos	14
3.7.1. DeepWalk	14
3.7.2. Node2Vec	14
3.8. Grafos espaciotemporales	15
4. Herramientas y datos utilizados	17
4.1. PyTorch	17
4.1.1. PyTorch Geometric	17
4.2. Wandb	18
4.3. MUTAG	18
4.4. Joint Attention in Autonomous Driving	19
4.4.1. Sumario	20
4.5. KNIME	21
5. Diseño del sistema y metodología	22
5.1. Diseño del grafo y representación de datos	22
5.1.1. Grafos espaciales	22
5.1.2. Grafos temporales	23
5.1.3. Sliding Window	23
5.1.4. Caso MultiPeaton	24
5.2. Pipeline experimental	25
5.3. Modelos GNN utilizados	25
5.3.1. Graph Convolutional Networks	26
5.3.2. Graph Attention Networks	26
5.3.3. GraphConv	26
5.4. Configuración experimental e hiperparámetros	27
5.5. Seguimiento experimental con Wandb	28
5.6. Seguimiento en GitHub	30

6. Pruebas y resultados	33
6.1. Resultados sobre MUTAG	33
6.2. Resultados sobre JAAD	36
6.2.1. Comparativa según el diseño del grafo	36
6.2.2. Comparativa según el tipo de datos	38
6.3. Comparativa de arquitecturas GNN	42
6.4. Análisis de robustez y semillas	42
6.5. Modelado avanzado	45
6.5.1. Modelado temporal	45
6.5.1.1. Conexión temporal básica	45
6.5.1.2. Conexión temporal completa	48
6.5.1.3. Ventanas deslizantes	50
6.5.2. Modelado multipeatón	52
6.5.2.1. Experimento multipeatón básico	52
6.5.2.2. Experimento multipeatón extendido	54
7. Discusión	57
7.1. Interpretación de resultados	57
7.2. Limitaciones del dataset	58
7.2.1. Revisión de muestras del dataset	58
7.2.2. Análisis del dataset	63
8. Conclusiones y futuros trabajos	68
8.1. Conclusiones	68
8.2. Futuros trabajos	69
Bibliografía	70

Índice de figuras

1.1. Complejidad en un entorno vial urbano	1
2.1. Representación de una escena vial basada en grafos	5
2.2. Conflictos en escena vial con peatones y vehículos	5
3.1. Técnicas de ML	7
3.2. Mapa del metro de Madrid como ejemplo práctico de grafo	8
3.3. Red neuronal básica	10
3.4. Grafo con la matriz que lo describe	10
3.5. Algoritmo de agregación entre vecinos	11
3.6. Grafo social en Facebook	11
3.7. Ilustración de semi-supervised node classification	12
3.8. Representación de una molécula en forma de grafo	13
3.9. Estrategias BFS y DFS a partir de un nodo	15
3.10. Datos de tráfico con estructura de grafo	16
4.1. Grafo adaptado con los datos de JAAD	20
5.1. Representación conceptual de los tres grafos espaciales	23
5.2. Representación conceptual del primer grafo espaciotemporal	23
5.3. Representación conceptual del segundo grafo espaciotemporal	24
5.4. Representación conceptual del tercer grafo espaciotemporal	24
5.5. Esquema de las multi capas en GCN	26
5.6. Ilustración de las múltiples cabezas en GAT	27
5.7. Ilustración del <i>pipeline</i> utilizado	28
5.8. Ejemplo de resultados en Wandb	30
5.9. Ejemplo de <i>commits</i> en GitHub	31
5.10. Tablero de planificación en GitHub	31

5.11. <i>Insights</i> en GitHub	32
6.1. Resultados sobre MUTAG usando GCN	34
6.2. Resultados sobre MUTAG usando GAT	35
6.3. Resultados sobre MUTAG usando GraphConv	35
6.4. Resultados utilizando el primer grafo	37
6.5. Resultados utilizando el segundo grafo	37
6.6. Resultados utilizando el tercer grafo	38
6.7. Resultados utilizando el dataset numérico para las distintas arquitecturas	39
6.8. Resultados utilizando el dataset lingüístico para las distintas arquitecturas	41
6.9. Resultados utilizando el grafo espaciotemporal básico	47
6.10. Resultados utilizando el grafo espaciotemporal completo	49
6.11. Resultados utilizando las ventanas deslizantes	51
6.12. Resultados utilizando el modelo multipeatón básico	53
6.13. Resultados utilizando el modelo multipeatón extendido	55
7.1. Muestra 16	59
7.2. Muestra 17	59
7.3. Muestra 147	60
7.4. Muestra 407	60
7.5. Muestra 527	61
7.6. Muestra 3188	62
7.7. Muestra 3190	62
7.8. Primera parte del flujo del análisis en KNIME	64
7.9. Segunda parte del flujo del análisis en KNIME	65
7.10. Ecuación del grado de conflicto	66
7.11. Flujo del análisis utilizando <i>Decision Tree</i> en KNIME	67

Índice de tablas

3.1. Tipos de tareas en grafos	13
5.1. Hiperparámetros configurables de los modelos GNN	29
6.1. Configuraciones para GCN	33
6.2. Configuraciones para GAT	34
6.3. Configuraciones para GraphConv	34
6.4. Comparativa de resultados con el dataset MUTAG	36
6.5. Comparativa de resultados con los tres grafos	38
6.6. Comparativa de resultados con los dos tipos de datasets	40
6.7. Comparativa de semillas en GCN	43
6.8. Resultados estadísticos con GCN	43
6.9. Comparativa de semillas en GAT	44
6.10. Resultados estadísticos con GAT	44
6.11. Comparativa de semillas en GraphConv	44
6.12. Resultados estadísticos con GraphConv	44
6.13. Comparativa de resultados con el grafo espaciotemporal básico	45
6.14. Comparativa de resultados con el grafo espaciotemporal completo	48
6.15. Comparativa de resultados con las ventanas deslizantes	50
6.16. Comparativa de resultados con el modelo multipeatón básico	53
6.17. Comparativa de resultados con el modelo multipeatón extendido	55

Lista de acrónimos

BFS	Breadth-first Sampling.
CNN	Convolutional Neural Networks.
DFS	Depth-first Sampling.
DL	Deep Learning.
GAT	Graph Attention Networks.
GCN	Graph Convolutional Networks.
GNN	Graph Neural Networks.
IA	Inteligencia Artificial.
JAAD	Joint Attention in Autonomous Driving.
KNIME	Konstanz Information Miner.
ML	Machine Learning.
PLN	Procesamiento del Lenguaje Natural.
PyG	PyTorch Geometric.
ReLU	Unidad Lineal Rectificada.
SAAC	Sistemas Avanzados de Asistencia al Conductor.
SCA	Sistemas de Conducción Automatizada.
STGCN	Space-Time Graph Convolutional Network.
Wandb	Weights & Biases.

Resumen

En este trabajo se ha llevado a cabo un estudio de predicción de la intención de los usuarios en contextos viales utilizando Redes neuronales basadas en grafos (GNN), explorando distintas representaciones espaciales, temporales y espaciotemporales. Se han evaluado varias arquitecturas GNN y configuraciones experimentales sobre diversas secuencias de un dataset, analizando su impacto en el rendimiento y estabilidad del modelo. Además, se han estudiado las limitaciones propias del dataset y se ha abordado el enfoque multipeatón. Por último, se proponen y discuten posibles líneas de mejora y trabajo futuro.

Palabras clave: redes neuronales basadas en grafos, clasificación de intención, grafos espaciotemporales, multipeatón.

Abstract

In this paper, a study on user intention prediction in road traffic scenarios has been conducted using Graph Neural Networks (GNN), focusing on the analysis of different spatial, temporal, and spatio-temporal graph representations. Through a series of experiments, multiple GNN architectures and experimental configurations have been systematically evaluated, assessing their impact on model performance and robustness. Furthermore, inherent dataset limitations are analyzed, and a multi-pedestrian modeling approach is addressed. Finally, potential improvements and future research directions are proposed and discussed.

Keywords: graph neural networks, intent classification, spatiotemporal graphs, multi-pedestrian.

Capítulo 1

Introducción

En esta primera sección se ha comentado la motivación del trabajo, así como los objetivos y la organización de éste.

1.1. Motivación y contexto

En la actualidad, la conducción autónoma está a la orden del día. Según la Real Academia Española, una de las definiciones de **autonomía** es la “condición de quien, para ciertas cosas, no depende de nadie”, concepto que resulta especialmente relevante en el ámbito de los Sistemas de Conducción Automatizada (SCA) [1].

En este contexto, un sistema autónomo ha de ser capaz de percibir el entorno que le rodea, extraer e interpretar la información disponible y tomar ciertas decisiones de una manera independiente y segura, incluyendo escenarios dinámicos e inciertos que requieran de una respuesta rápida.

Los entornos viales presentan una gran complejidad debido a la coexistencia de agentes (véase Figura 1.1) cuyos comportamientos pueden ser variables y difíciles de predecir. En concreto, los peatones representan uno de los usuarios más impredecibles, ya que sus decisiones no siguen unas normas predefinidas y pueden verse influenciadas por distintos factores contextuales y temporales.

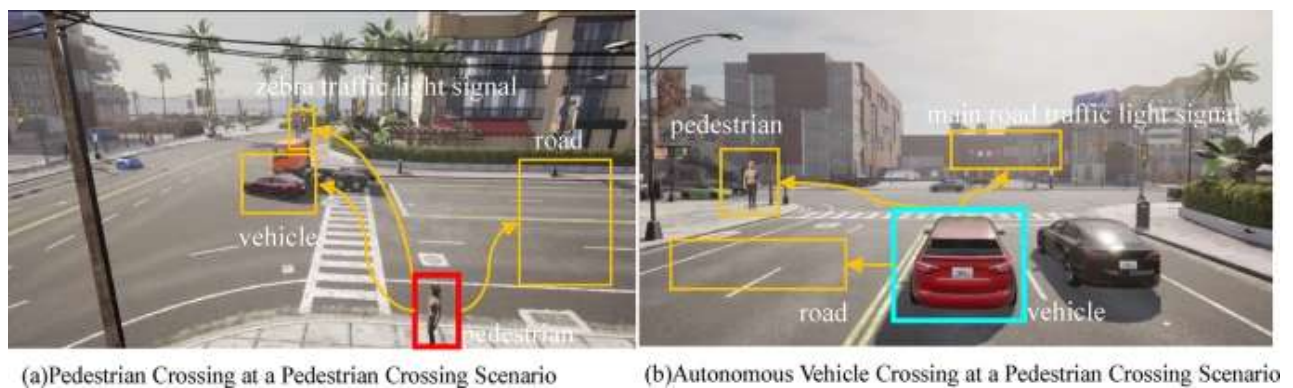


Figura 1.1: Complejidad en un entorno vial urbano

Es por ello que la capacidad de predecir la intención de los usuarios en la vía se convierte en una necesidad clave para poder mejorar la seguridad y fiabilidad de los SCA. Una estimación a

tiempo y con precisión permite anticipar futuros comportamientos, adaptando el control del vehículo y reduciendo el riesgo en situaciones potencialmente peligrosas.

La comprensión del comportamiento de los usuarios de la vía resulta fundamental para garantizar una mayor seguridad a la hora de tomar decisiones en los SCA. A pesar de los recientes avances en los Sistemas Avanzados de Asistencia al Conductor (SAAC), sigue siendo un desafío lograr una predicción fiable de la intención de los peatones, en especial, en escenarios urbanos, ya que presentan mayor complejidad y dinamismo. La variabilidad del comportamiento humano y la interacción entre múltiples agentes introduce una gran incertidumbre, ya que dificulta alcanzar unos niveles elevados de automatización consistente en diversos contextos viales [2].

En esta línea, la predicción de la intención de los peatones (por ejemplo, la intención de cruce) genera un problema de gran relevancia en el campo de la percepción del entorno. Esta clase de predicción no depende exclusivamente de la posición del peatón, sino que existen otras características relevantes como el comportamiento previo de éste, la dinámica temporal de la escena o la interacción con otros usuarios de la vía.

Modelar este tipo de escenarios de una manera adecuada requiere ser capaz de poder capturar tanto relaciones espaciales entre los distintos usuarios como su evolución temporal. En este contexto, los grafos permiten representar dichas interacciones, describiendo de una forma explícita las dependencias entre peatones, vehículos y entorno. Sobre esta representación, las GNN han demostrado ser una poderosa herramienta para el aprendizaje de patrones complejos en dominios relacionales y dinámicos [3].

1.2. Objetivos

El objetivo de este trabajo consiste en la evaluación de técnicas basadas en GNN para la estimación de la intención de los usuarios de la vía, como pueden ser peatones o vehículos, en entornos viales. Para ello se empleará una base de datos ya existente para realizar los entrenamientos sobre una serie de modelos.

Una vez entrenados estos modelos, se realizará una comparación del rendimiento de las GNN con técnicas estándar de Machine Learning (ML), con el fin de analizar el potencial de las GNN frente a métodos tradicionales.

1.3. Organización de la memoria

La memoria se encuentra dividida en ocho capítulos:

1. **Introducción:** En este primer capítulo se expone la motivación del trabajo, los objetivos y la organización propia de la memoria.
2. **Estado del arte:** A continuación, se comenta el estado actual de los estudios sobre la predicción de la intención de peatones, así como los enfoques sobre grafos y temporales.

3. **Marco teórico:** Se describen los conceptos fundamentales utilizados en el trabajo, como los grafos, el ML, las GNN, grafos espaciotemporales y técnicas de *embedding* en grafos.
4. **Dataset y análisis de datos:** Se presentan los datasets utilizados (MUTAG y JAAD), así como las herramientas utilizadas para el desarrollo del trabajo.
5. **Diseño del sistema y metodología:** En este capítulo se detalla la construcción del grafo, la representación de los datos, el *pipeline* experimental, los modelos GNN empleados (GCN, GAT y GraphConv), la configuración experimental, las pruebas con hiperparámetros y el seguimiento de experimentos mediante Wandb.
6. **Pruebas y resultados:** Aquí se exponen las pruebas y los resultados obtenidos utilizando los datasets MUTAG y JAAD, la comparativa entre arquitecturas GNN, el análisis de robustez frente a diferentes semillas, y el modelado avanzado, incluyendo temporal y multiépoca.
7. **Discusión:** Se interpretan los resultados obtenidos, identificando las limitaciones de los modelos y de los datasets utilizados.
8. **Conclusiones y futuro trabajo:** Por último, se presentan las principales conclusiones del trabajo y se proponen posibles líneas de investigación y mejoras futuras.

Capítulo 2

Estado del arte

2.1. Predicción de intención de peatones

Los estudios sobre la predicción de la intención de los peatones se han convertido en un área de investigación clave dentro de los SCA, debido al gran impacto que producen en la seguridad vial. La alta variabilidad del comportamiento de los peatones dificulta su modelado y su predicción.

Los primeros enfoques se basaron en el uso de características geométricas y cinemáticas como pueden ser la posición, la velocidad o la trayectoria del peatón, empleando modelos probabilísticos o técnicas de ML. Posteriormente, gracias a las técnicas de Deep Learning (DL), surgen nuevos modelos basados en Convolutional Neural Networks (CNN), capaces de capturar patrones espaciales y temporales a partir de una secuencia de imágenes [4].

Sin embargo, se ha demostrado que la intención de cruce por parte de un peatón no depende exclusivamente de su estado individual, sino también del contexto de la escena y de la interacción con otros agentes como vehículos [5]. Esta observación ha motivado al desarrollo de modelos de mayor complejidad capaces de integrar información visual, contextual y temporal, dando lugar a modelos multimodales capaces de mejorar el rendimiento en contextos urbanos.

2.2. Enfoques basados en grafos y Graph Neural Networks

Con el objetivo de modelar relaciones entre peatón y entorno, los grafos son considerados como una representación natural para describir escenas viales complejas (véase Figura 2.1).

Sobre esta estructura, las GNN han demostrado ser especialmente adecuadas para el aprendizaje de patrones complejos, permitiendo la propagación y agregación de información entre nodos conectados [3]. Este tipo de modelos es ampliamente utilizado para tareas de clasificación de grafos o predicción de enlaces.

Dentro de la predicción de intención de peatones, las GNN son utilizadas para capturar interacciones entre agentes explícitamente. Un enfoque basado en grafos permite razonar sobre las relaciones dinámicas entre peatones y vehículos, logrando mejoras significativas en JAAD [6]. Además, también se ha demostrado que la incorporación de información contextual en los grafos mejora la robustez y generalización frente a enfoques individuales [2].

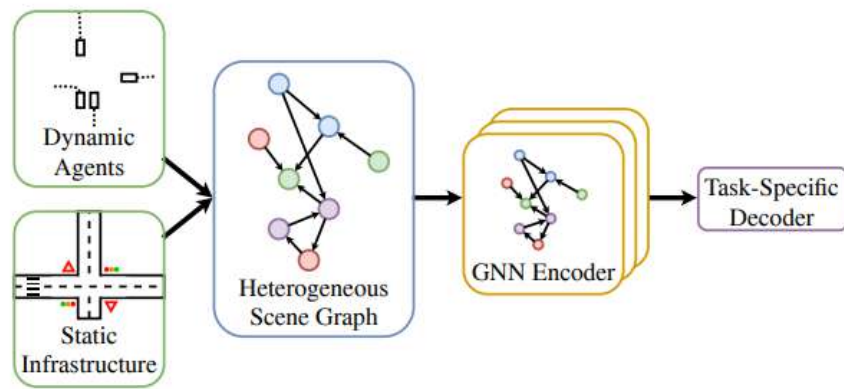


Figura 2.1: Representación de una escena vial basada en grafos

2.3. Modelado temporal y multipeatón

Además de las relaciones espaciales, el modelado temporal es fundamental en la predicción de intención, ya que el comportamiento de los peatones evoluciona de una manera continua a lo largo del tiempo. Por eso, las representaciones basadas en grafos se extienden a modelos espaciotemporales, donde se capturan tanto las interacciones entre los agentes como su evolución temporal.

Las Space-Time Graph Convolutional Network (STGCN) permiten modelar una secuencia de grafos ya no solo conectando nodos en el espacio, sino también a lo largo del tiempo, facilitando el aprendizaje de situaciones complejas, siendo un enfoque de gran interés en escenarios multipeatón (véase Figura 2.2), donde el aspecto social entre peatones puede afectar considerablemente a la acción individual [7].

Por otro lado, la incorporación de múltiples peatones en un mismo grafo permite capturar comportamientos colectivos, mejorando así la capacidad predictiva. Además, la integración de mecanismos de atención permite ponderar la relevancia de cada interacción en función del contexto vial [8].

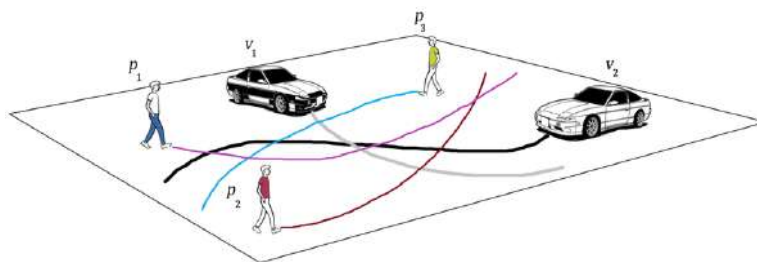


Figura 2.2: Conflictos en escena vial con peatones y vehículos [9]

Por estas razones, estos avances dejan en evidencia que el uso de STGCN y enfoques multipeatón constituyen una nueva línea de investigación en desarrollo para la predicción de intención de peatones en situaciones viales complejas.

Capítulo 3

Marco teórico

El objetivo de este capítulo es explicar los distintos conceptos teóricos de los que se sustenta el trabajo.

3.1. Inteligencia Artificial

La IA puede definirse como la ciencia e ingeniería capaz de crear máquinas inteligentes, especialmente programas de ordenador inteligentes [10]. De una forma más funcional, la IA puede entenderse como el estudio de agentes racionales que pueden percibir y actuar sobre un entorno para maximizar el cumplimiento de sus objetivos [11].

Estas técnicas se basan en áreas como la búsqueda y resolución de problemas, la lógica, el razonamiento probabilístico o el aprendizaje. Dentro de este marco, surgen enfoques como el Machine Learning (ML) y el Deep Learning (DL), permitiendo a los sistemas aprender representaciones complejas gracias a los datos.

3.2. Machine Learning

En este contexto, el ML surge como una de las grandes ramas de la IA, centrándose en el desarrollo de algoritmos capaces de aprender patrones a partir de unos datos, sin la necesidad de tener que ser programados de forma explícita para una tarea. Un programa aprende de la experiencia con respecto a una clase de tareas y una medida de rendimiento si éste último mejora con la experiencia [12].

El objetivo del ML es construir modelos que sean capaces de generalizar un conocimiento aprendido a partir de unos ejemplos previos, realizando así predicciones o desarrollando una toma de decisiones sobre datos nuevos. Tradicionalmente, las técnicas de ML se clasifican en función del tipo de supervisión (véase Figura 3.1). Por ejemplo, en el aprendizaje supervisado, el modelo se entrena con datos etiquetados para aprender a predecir resultados o clasificar los nuevos datos sin etiquetar. Por otra parte, el aprendizaje no supervisado descubre estructuras presentes en los datos sin tener etiquetas explícitas. No todo es blanco o negro, también existen enfoques intermedios como el aprendizaje semi-supervisado, o enfoques alternativos como el aprendizaje por refuerzo, donde el agente imita un proceso de prueba y error para lograr sus objetivos.

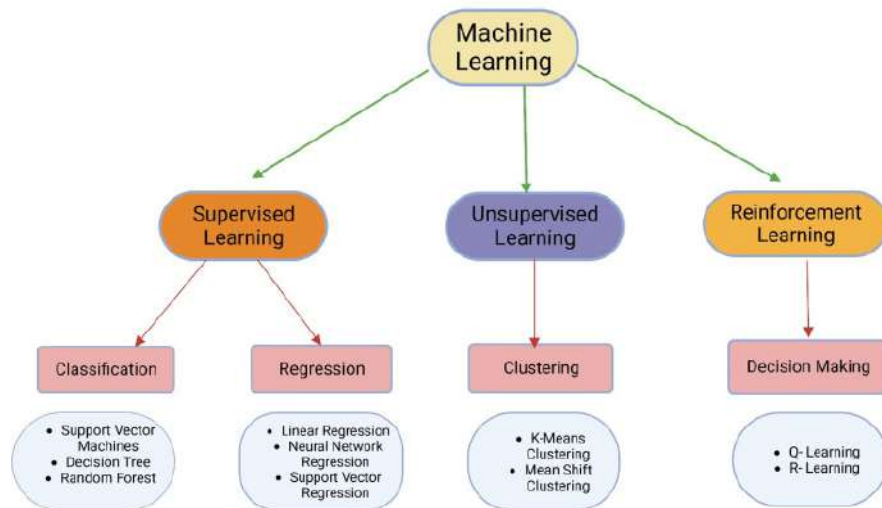


Figura 3.1: Técnicas de ML [13]

En un problema real, como puede ser el entorno vial, el rendimiento de modelos de ML depende en gran medida de la calidad de los datos de entrada. Cuando estos datos representan estructuras complejas, o contienen dependencias entre múltiples entidades, estas representaciones tradicionales pueden resultar limitadas. Gracias a esta necesidad, se han desarrollado técnicas más elaboradas capaces de trabajar sobre estructuras de mayor complejidad, como los grafos, dando lugar a una investigación activa entre el ML y la teoría de grafos.

3.3. Grafos para el Machine Learning

En muchos problemas reales, los datos no son presentados de forma independiente, sino que se relacionan explícitamente entre múltiples entidades que influyen de manera directa en el comportamiento del sistema, por lo que los modelos tradicionales acaban siendo insuficientes, ya que no son capaces de capturar esta compleja estructura de datos y sus dependencias.

Los grafos son una herramienta matemática capaz de modelar información estructurada, ya que permiten representar entidades como nodos y sus relaciones como aristas. Esta representación resulta fundamental en ámbitos donde las interacciones entre elementos desempeñan un papel clave, como las redes sociales, los entornos viales, la biología computacional, o los sistemas de transporte [14]. Un ejemplo de la aplicación de grafos en la vida cotidiana es un mapa de metro, donde las estaciones representan los vértices y la propia línea de metro las aristas, conectando dichas estaciones (véase Figura 3.2).

Los parámetros fundamentales de un grafo son:

- **V:** conjunto de vértices (o nodos).
- **E:** conjunto de aristas (o enlaces).

Formalmente, la ecuación de un grafo se puede expresar como $G = (V, E)$.



Figura 3.2: Mapa del metro de Madrid como ejemplo práctico de grafo

En el contexto del ML, el uso de grafos permite integrar información relacional de forma explícita en el aprendizaje, facilitando la comprensión de patrones latentes ya no solo en función de las características individuales, sino también de su posición y conexión dentro de la estructura global. Esta representación supone un reto, ya que los grafos son de naturaleza irregular, debido a su tamaño no euclidiano y variable, dificultando su aplicación en tareas basadas en técnicas tradicionales de aprendizaje automático [3].

Estas limitaciones son el motivo del desarrollo de nuevos métodos capaces de operar sobre información estructurada en forma de grafo, dando lugar a una investigación conjunta entre ML y la teoría de grafos. Sobre esta base surgen nuevos enfoques avanzados, como el Deep Learning (DL) sobre grafos, que permite aprender representaciones de alto nivel a partir de la propia estructura y atributos del grafo [3].

3.4. Deep Learning

El DL surge como una extensión del ML, siendo orientado al aprendizaje automático de representaciones jerárquicas a partir de un gran volumen de datos. A diferencia de los modelos clásicos, donde las características relevantes son diseñadas a mano, los modelos de DL son capaces de aprender estas representaciones automáticamente gracias a la composición de múltiples capas de procesamiento no lineal [15].

Este enfoque ha demostrado un gran rendimiento en tareas de visión por computador, Procesamiento del Lenguaje Natural (PLN) o reconocimiento de patrones complejos, donde las relaciones entre datos no son triviales y tienen una alta dimensionalidad. Gracias a esta capacidad de modelar funciones no lineales, el DL se ha convertido en una rama de la IA especialmente adecuada para

problemas complejos y dinámicos.

3.4.1. Redes neuronales

Las redes neuronales artificiales forman parte de una base fundamental del DL, estando inspiradas en el funcionamiento del sistema nervioso biológico del ser humano (véase Figura 3.3). Una red neuronal está compuesta por unidades elementales llamadas neuronas, organizadas en capas, donde procesan la información mediante combinaciones lineales y funciones de activación no lineales [16].

En términos generales, una red neuronal se encuentra estructurada en una capa de entrada, una o varias capas ocultas y una capa de salida. Cada neurona recibe un conjunto de entradas de otras, aplicando una función de activación donde se introduce no linealidad en el modelo, permitiendo así aprender relaciones complejas entre los datos. Algunas de las funciones de activación más comunes son las siguientes [17]:

- **Función lineal:**

$$f(x) = \begin{cases} -1 & \text{si } x \leq -\frac{1}{a} \\ a \cdot x & \text{si } -\frac{1}{a} < x < \frac{1}{a} \\ 1 & \text{si } x \geq \frac{1}{a} \end{cases}$$

donde $x = \text{Gini} - \Theta_i$ y $a > 0$.

- **Función sigmoidea:**

$$f(x) = \left\{ \frac{1}{1+e^{-gx}} \right.$$

donde $x = \text{Gini} - \Theta_i$.

- **Función tangente hiperbólica:**

$$f(x) = \left\{ \frac{e^{gx} - e^{-gx}}{e^{gx} + e^{-gx}} \right.$$

donde $x = \text{Gini} - \Theta_i$.

El proceso de aprendizaje se realiza ajustando los pesos de la red mediante diversos algoritmos de optimización, como el descenso por gradiente, minimizando los errores entre los resultados previstos y los reales. Gracias a este mecanismo, las redes neuronales pueden aproximar funciones complejas, generalizando a datos no vistos.

Sin embargo, estas arquitecturas neuronales tradicionales fueron diseñadas para trabajar sobre datos estructurados en forma de vectores o matrices, limitando su aplicación en ámbitos donde la información se representa de manera no euclidiana, como pueden ser los grafos. Es por ello del desarrollo de arquitecturas específicas para extender el DL a este tipo de representaciones, dando lugar a las redes neuronales basadas en grafos (GNN).

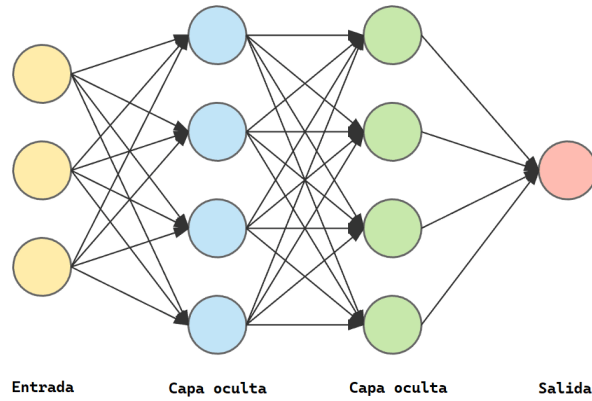


Figura 3.3: Red neuronal básica

3.5. Redes neuronales basadas en grafos

Las Graph Neural Networks (GNN) surgen como una extensión del DL orientadas a trabajar directamente sobre datos estructurados en forma de grafo. A diferencia de otras arquitecturas neuronales, las GNN fueron diseñadas para poder trabajar con datos cuya estructura no es regular, donde son las relaciones entre entidades las que adaptan un rol fundamental a la hora de interpretar los datos [3].

En un grafo, la información no se encuentra exclusivamente en los atributos individuales de un nodo, sino también en la estructura de conexiones e interacciones entre nodos vecinos. Son las GNN las que permiten incorporar de una forma explícita información relacional en el aprendizaje, aprendiendo representaciones vectoriales de los nodos, aristas o incluso del grafo completo, teniendo en cuenta tanto las características individuales como el contexto estructural.

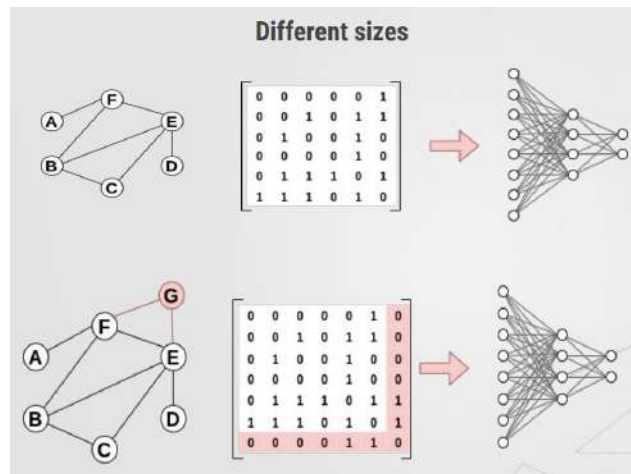


Figura 3.4: Grafo con la matriz que lo describe [18]

El principio fundamental de las GNN es el mecanismo de propagación de mensajes (*message passing*), donde cada nodo actualiza sus características mediante la información procedente de sus vecinos, en un intercambio iterativo. Este intercambio de información permite que la información se propague a través del grafo, por lo que cada nodo puede capturar dependencias de orden superior [19]. Al cambiar el orden de los nodos, también cambia el de la matriz, por lo que el dominio no es

euclidiano y, al añadir un nodo, cambia el tamaño de la matriz (véase Figura 3.4).

Una capa de una GNN puede definirse como una operación de agregación y actualización, donde los nodos aprenden iterativamente en función de sus vecinos (véase Figura 3.5). Gracias a este enfoque, se puede aprender representaciones invariables al orden de nodos y adaptables a cualquier tamaño, superando así las limitaciones de los modelos tradicionales de DL.

Algorithm 1: Neighborhood-aggregation encoder algorithm. Adapted from [29].

Input : Graph $\mathcal{G}(\mathcal{V}, \mathcal{E})$; input features $\{\mathbf{x}_v, \forall v \in \mathcal{V}\}$; depth K ; weight matrices $\{\mathbf{W}^k, \forall k \in [1, K]\}$; non-linearity σ ; differentiable aggregator functions $\{\text{AGGREGATE}_k, \forall k \in [1, K]\}$; neighborhood function $\mathcal{N} : v \rightarrow 2^{\mathcal{V}}$

Output: Vector representations $\mathbf{z}_v$ for all $v \in \mathcal{V}$

```

1  $\mathbf{h}_v^0 \leftarrow \mathbf{x}_v, \forall v \in \mathcal{V}$ ;
2 for  $k = 1 \dots K$  do
3   for  $v \in \mathcal{V}$  do
4      $\mathbf{h}_{\mathcal{N}(v)}^k \leftarrow \text{AGGREGATE}_k(\{\mathbf{h}_u^{k-1}, \forall u \in \mathcal{N}(v)\})$ ;
5      $\mathbf{h}_v^k \leftarrow \sigma(\mathbf{W}^k \cdot \text{COMBINE}(\mathbf{h}_v^{k-1}, \mathbf{h}_{\mathcal{N}(v)}^k))$ 
6   end
7    $\mathbf{h}_v^k \leftarrow \text{NORMALIZE}(\mathbf{h}_v^k), \forall v \in \mathcal{V}$ 
8 end
9  $\mathbf{z}_v \leftarrow \mathbf{h}_v^K, \forall v \in \mathcal{V}$ 

```

Figura 3.5: Algoritmo de agregación entre vecinos [20]

Gracias a estas propiedades, las GNN han mostrado un gran rendimiento en múltiples tareas relacionadas con grafos, como la clasificación de nodos, la predicción de enlaces y la clasificación de grafos completos. Un claro ejemplo es el sistema de recomendación de amigos en redes sociales (véase Figura 3.6), donde las GNN ayudan a predecir posibles conexiones entre usuarios gracias a interacciones previas y características similares [20].

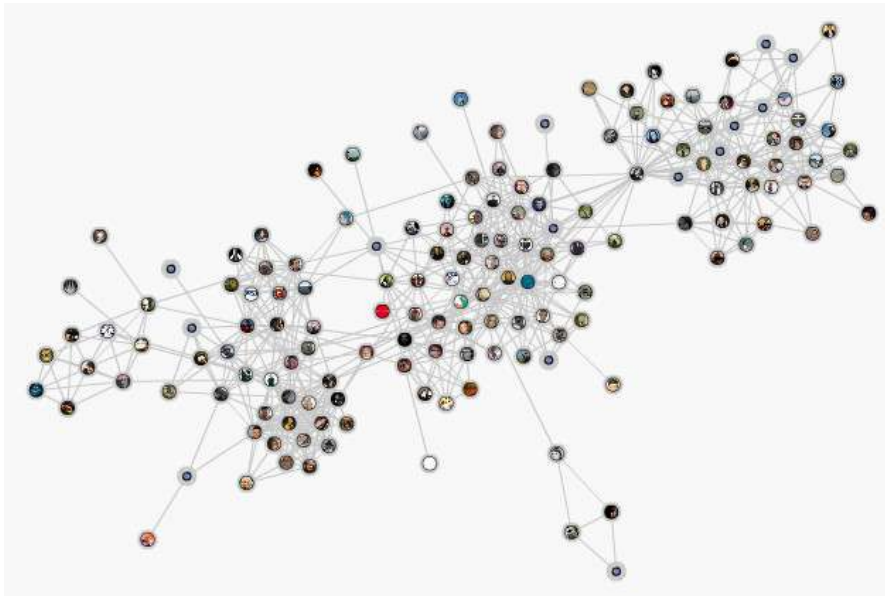


Figura 3.6: Grafo social en Facebook

En el contexto de este trabajo, las GNN son especialmente adecuadas para la predicción de intención, ya que permiten modelar explícitamente las interacciones entre peatones, vehículo y entorno, integrando información espacial y temporal en una misma arquitectura estructurada.

3.6. Tipos de tareas en grafos

Mediante el uso de redes neuronales, se pueden abordar distintas tareas en función del nivel de estructura del grafo sobre el que se va a hacer la predicción. Esta distinción permite diferenciar si la predicción se realiza para cada nodo individual, para los vértices entre nodos, o para propiedades del grafo completo.

3.6.1. Node-level tasks

Las tareas de tipo *node-level* tienen como objetivo clasificar los nodos de un grafo. Se busca estimar una etiqueta para cada nodo individual, aprovechando sus características propias y la información procedente de los vecinos.

Un ejemplo de esta categoría es la clasificación de nodos semisupervisada (*semi-supervised node classification*) (véase Figura 3.7), donde se proporciona un único grafo con algunos nodos ya etiquetados, aprendiendo a clasificar éstos durante el entrenamiento, para luego generalizar a los nodos sin etiquetar [21].

Este tipo de tarea resulta de gran utilidad cuando se quiere estimar propiedades de entidades individuales en un sistema interconectado. En el contexto del trabajo, el uso de *node-level* corresponde a la predicción de intención de un peatón, donde cada nodo representa a un usuario, realizando la predicción para cada uno de ellos.

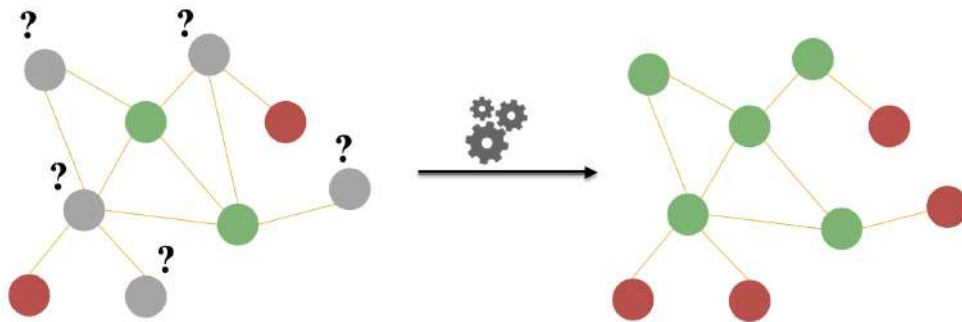


Figura 3.7: Ilustración de semi-supervised node classification

3.6.2. Edge-level tasks

Las tareas de tipo *edge-level* se centran en realizar predicciones sobre las conexiones entre pares de nodos. A diferencia de *node-level*, este tipo se centra en estimar si una arista debería existir, y en caso afirmativo, clasificar una propiedad a dicha conexión.

El tipo más común es la predicción de enlaces (*link prediction*), donde, dado un grafo, se intenta predecir si debería haber una arista entre dos nodos que no están directamente conectados. basándose en la estructura del grafo y las características de los nodos implicados [21].

Otra tarea a nivel de *edge-level* es la clasificación de aristas ya existentes, donde se asigna una categoría o valor a cada relación, siendo útil en campos como la química computacional, o en interacciones de entidades en sistemas complejos.

3.6.3. Graph-level tasks

Por último, las tareas de tipo *graph-level* consisten en realizar una predicción global sobre todo el grafo, con el fin de lograr una etiqueta común que represente una propiedad del conjunto de nodos y aristas, siendo de especial interés para caracterizar sistemas complejos, más allá de sus elementos individuales.

Dentro de *graph-level* se realiza la clasificación de grafos (*graph classification*), donde se busca asignar una clase a cada grafo completo dentro de un conjunto de ejemplos, siendo de especial interés en la predicción de relaciones moleculares (véase Figura 3.8), donde las moléculas se representan como grafos [21].

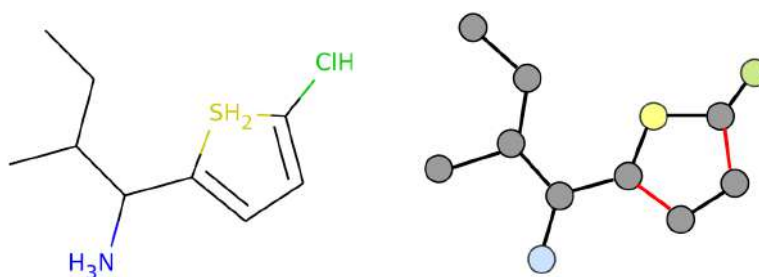


Figura 3.8: Representación de una molécula en forma de grafo [21]

3.6.4. Resumen de tipos de tareas en grafos

Para resumir estas tres tareas, se adjunta la siguiente tabla:

Tipo de tarea	Unidad	Objetivo	Ejemplo	Aplicación común
<i>Node-level tasks</i>	Nodo	Predecir una etiqueta para cada nodo	<i>Semi-supervised node classification</i>	Predicción de intención de un peatón
<i>Edge-level tasks</i>	Arista	Clasificar una relación entre dos nodos	<i>Link prediction</i>	Predicción de interacciones químicas
<i>Graph-level tasks</i>	Grafo	Asignar una etiqueta global a un grafo	<i>Graph classification</i>	Predicción de relaciones moleculares

Tabla 3.1: Tipos de tareas en grafos

3.7. Técnicas de embedding en grafos

Una de las principales dificultades a la hora de aplicar técnicas de ML o DL sobre grafos es transformar la estructura irregular propia de un grafo en representaciones vectoriales que puedan ser procesadas por modelos matemáticos. Es aquí donde surgen las técnicas de *embedding* en grafos (*graph embedding*), cuyo objetivo es aprender representaciones de baja dimensión para nodos, aristas y grafos completos, preservando así información estructural y propiedades del grafo de origen [22].

Estas técnicas permiten capturar propiedades como la proximidad entre nodos, la similitud estructural o la pertenencia a un grupo de nodos, facilitando así a tareas como la clasificación de nodos, la predicción de enlaces o el *clustering*, es decir, la agrupación de nodos similares según un criterio de similitud. Entre estas representaciones destacan *DeepWalk* y *Node2Vec*, las cuales sentaron las bases del aprendizaje en representaciones de grafos antes de la aparición de las GNN.

3.7.1. DeepWalk

DeepWalk fue una de las primeras propuestas para el aprendizaje de *embeddings* en grafos, el cual consiste en aprender representaciones de los vértices en un espacio vectorial continuo que codifica sus relaciones estructurales, generalizando técnicas de modelado del lenguaje y aprendizaje no supervisado mediante el uso de paseos aleatorios (*random walks*) tratados como secuencias [23].

Cada paseo aleatorio se interpreta como una frase, siendo los nodos palabras, y usando el modelo *SkipGram* con el objetivo de aprender representaciones vectoriales, maximizando la probabilidad de coocurrencia entre las palabras (nodos) cercanas en dichas secuencias.

Es por esto que *DeepWalk* es capaz de capturar información estructural del grafo, logrando la proximidad entre nodos que aparecen con frecuencia en un mismo recorrido. Por otra parte, al basarse en caminos aleatorios, esta técnica presenta limitaciones para capturar estructuras de mayor complejidad, como por ejemplo la similitud entre nodos que no están directamente conectados.

3.7.2. Node2Vec

Por otra parte, *Node2Vec* surge con el objetivo de mejorar la flexibilidad en la exploración del grafo durante la generación de los paseos aleatorios. A diferencia de *DeepWalk*, esta nueva técnica introduce una técnica de *random walks* sesgados con el fin de explorar diversos vecindarios de nodos, todo gracias a dos parámetros clave: el retorno p y la entrada-salida q [24].

Gracias a estos dos nuevos parámetros, se puede intercalar entre dos técnicas de exploración (véase Figura 3.9):

1. **Breadth-first Sampling (BFS):** En este tipo de exploración, el vecindario N_S queda restringido a aquellos nodos que son vecinos inmediatos del nodo fuente.
2. **Depth-first Sampling (DFS):** En este otro caso de exploración, el vecindario consiste en nodos muestreados secuencialmente a distancias crecientes respecto del nodo fuente.

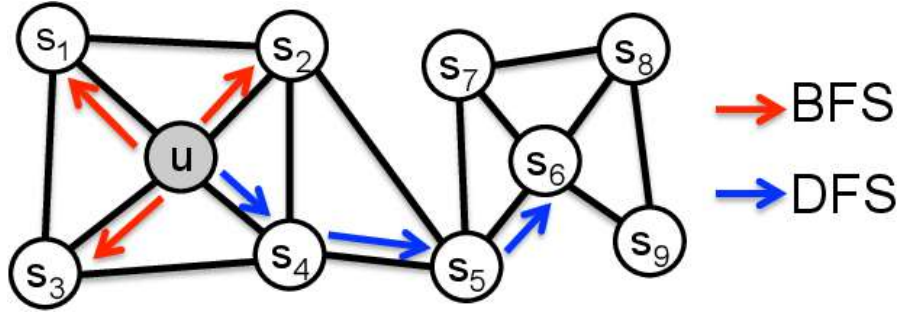


Figura 3.9: Estrategias BFS y DFS a partir de un nodo [24]

Es por este mecanismo por lo que *Node2Vec* tiene un mayor control sobre la información estructural a conservar en los *embeddings* aprendidos. Gracias a esta flexibilidad, *Node2Vec* se ha convertido en una técnica ampliamente utilizada en tareas como *link prediction* o *node classification*.

3.8. Grafos espaciotemporales

En un contexto realista, las relaciones entre entidades no solo dependen de una estructura espacial, sino que también dependen de una evolución temporal. En esta clase de escenarios, un grafo estático en el tiempo resulta insuficiente para modelar esta nueva dinámica temporal del sistema, ya que no capturan cambios temporales ni el proceso evolutivo de los nodos. Debido a esta necesidad nacen los grafos espaciotemporales, siendo una extensión natural de grafos que incorporan explícitamente una dimensión temporal [25].

Un grafo espaciotemporal permite representar tanto las relaciones espaciales entre entidades como la evolución de éstas a lo largo del tiempo. Por norma general, estos grafos se construyen mediante una secuencia ordenada de grafos, donde cada grafo representa un instante temporal, o bien mediante aristas temporales que conectan nodos en distintos instantes, capturando simultáneamente dependencias espaciales y temporales en una estructura [3].

Este tipo de grafo resulta de gran interés desde situaciones cotidianas como el análisis del tráfico (véase Figura 3.10) o sistemas de transporte hasta situaciones más complejas como la predicción de trayectorias o el modelado de comportamientos humanos, ya que las interacciones entre agentes cambian constantemente. Por ejemplo, en un contexto vial, no es conveniente analizar el estado de un peatón de forma aislada, ya que hay que tener en cuenta su evolución temporal y su relación con el entorno [7].

La combinación de estos grafos con técnicas de DL ha logrado arquitecturas específicas las cuales son capaces de aprender patrones complejos para estos datos, destacando las STGCN, los cuales integran mecanismos de convolución espacial sobre grafos temporales, capturando así dependencias tanto locales como globales en el espacio y el tiempo [7]. Gracias a estas capacidades, estas arquitecturas muestran un gran rendimiento en contextos de predicción temporal, detección de eventos o análisis de comportamiento dinámico.

En el caso concreto de este trabajo, el uso de grafos espaciotemporales resulta de gran interés para analizar la predicción de intención del peatón, ya que, aparte de modelar relaciones espaciales

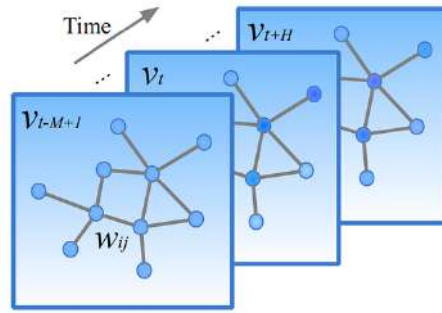


Figura 3.10: Datos de tráfico con estructura de grafo. Cada v_t indica un frame del estado actual del tráfico en el instante temporal t , el cual se registra en una matriz de datos con estructura de grafo. [7]

con otros usuarios, también se contempla la evolución temporal en dichas interacciones, consolidando una sólida base en desarrollo de modelos capaces de anticipar futuros comportamientos en contextos viales.

Capítulo 4

Herramientas y datos utilizados

Para poder desarrollar el sistema, se han utilizado diferentes herramientas y datos que serán detallados en este capítulo.

4.1. PyTorch

PyTorch es una librería de DL de código abierto destinada a proporcionar un equilibrio entre un alto rendimiento, facilidad de uso y flexibilidad en la comunicación. *PyTorch* estiliza la programación de una manera imperativa, es decir, las operaciones se ejecutan de forma inmediata, permitiendo así interactuar directamente con el flujo de ejecución [26].

A diferencia de otros marcos, una de las principales características de *PyTorch* es el uso de grafos dinámicos, facilitando la construcción y modificación de modelos durante la ejecución. Este *framework* es adecuado para trabajar con arquitecturas o estructuras de datos complejas. Otra ventaja de *PyTorch* es que permite calcular los gradientes eficientemente, por lo que simplifica el entrenamiento de redes neuronales.

La librería *PyTorch* sirve como base para la implementación de modelos de DL utilizados, debido a su manera de trabajo eficiente con estructuras dinámicas y temporales. Además, la flexibilidad y compatibilidad que ofrece con otras librerías convierte a este *framework* en el ideal para trabajar con técnicas de grafos.

4.1.1. PyTorch Geometric

PyTorch Geometric (PyG) es una biblioteca de *Python* desarrollada sobre *PyTorch* para escribir y entrenar fácilmente GNN para una amplia gama de aplicaciones relacionadas con datos estructurados. Consiste en varios métodos de DL en grafos y otras estructuras irregulares, también conocido como aprendizaje profundo geométrico. Además, incluye *mini-batch loaders* fáciles de usar para operar tanto en grafos pequeños como en gigantes. Tiene compatibilidad con:

1. **Múltiples GPU:** Es un soporte nativo para distribuir el entrenamiento en múltiples GPU, permitiendo escalar modelos de GNN de manera eficiente gracias a la paralelización de los datos.

2. **torch.compile**: Es un método que permite una ejecución más rápida gracias a la compilación en núcleos optimizados, sin requerir de grandes cambios en el código.
3. **DataPipe**: Integración de *DataPipes* integrados por PyG para procesar grandes conjuntos de datos en grafos.
4. **Datasets**: Contiene una colección de datasets con datos estándar y de referencia para el aprendizaje en grafos, útiles para tareas como *node classification*, *link prediction* o *graph classification*.
5. **Transformaciones útiles**: Es un conjunto de transformaciones ya predefinidas para el aprendizaje en grafos arbitrarios como en mallas 3D o nubes de puntos.

Una gran ventaja que ofrece PyG es la capacidad de manejar grafos independientemente de su tamaño y estructura, abstrayendo la complejidad a la representación de nodos, aristas y atributos. Además, cuenta con mecanismos optimizados para la propagación de mensajes y agregación entre nodos [27].

PyG sirve como herramienta base del trabajo, ya que permite construir y entrenar modelos basados en grafos, además de definir modelos espaciotemporales y aplicar técnicas de DL eficientemente. La propia flexibilidad y rendimiento de la biblioteca de *PyTorch* hace que PyG sea adecuada para el modelado de las interacciones entre usuarios en un escenario vial.

4.2. Wandb

Weights & Biases (Wandb) es una plataforma destinada al seguimiento, visualización y gestión de experimentos en proyectos de IA, facilitando la monitorización sistemática de entrenamientos, registrando y centralizando métricas, hiperparámetros, configuraciones y resultados de forma reproducible [28].

Esta herramienta permite analizar en tiempo real la evolución de distintos parámetros como la pérdida o la precisión, además de medidas personalizadas. También resulta de gran utilidad al almacenar los valores de los hiperparámetros utilizados, facilitando los procesos de ajuste de estos valores.

4.3. MUTAG

MUTAG es un conjunto de datos clásico utilizado como *benchmark* en la clasificación de grafos dentro del ML, y en concreto, en el estudio de modelos GNN. Este dataset modela compuestos químicos utilizando grafos, donde cada nodo representa un átomo y cada arista, un enlace químico entre átomos [3]. El objetivo de *MUTAG* es predecir la mutagenicidad de los compuestos, indicando el potencial para causar mutaciones en el ADN de *Salmonella typhimurium*.

Cada grafo en *MUTAG* incorpora información estructural y atributos sobre los nodos, como el tipo de átomo, convirtiendo a este dataset en el adecuado para evaluar la capacidad de modelos de aprendizaje en grafos. El ligero tamaño (188 compuestos) y la complejidad estructural han consolidado

a MUTAG como un dataset de referencia para comparar arquitecturas y técnicas de aprendizaje profundo geométrico [29].

En la práctica, MUTAG se encuentra integrado en PyG, facilitando así su uso en experimentos con grafos. Esta integración favorece a la adopción de MUTAG como un *benchmark* de validación para el desarrollo y análisis de modelos GNN. Gracias a estas características, MUTAG ha servido como dataset de referencia en este trabajo para validar el funcionamiento de las arquitecturas implementadas en un entorno controlado, sirviendo de antesala para datos más complejos.

4.4. Joint Attention in Autonomous Driving

Joint Attention in Autonomous Driving (JAAD) es un dataset especializado en el estudio de la atención conjunta en el contexto de la conducción autónoma. El objetivo principal de JAAD es analizar el comportamiento humano, tanto de peatones como de conductores, en situaciones de tráfico para analizar los factores que influyen en la intención del peatón a la hora de cruzar [30].

El dataset está compuesto por 346 vídeos grabados desde la perspectiva del vehículo, junto con unas anotaciones temporales, de comportamiento, de contexto, y atributos del peatón. Estas anotaciones permiten analizar la evolución temporal del comportamiento, convirtiendo a JAAD en un conjunto de datos idóneo para enfoques secuenciales y espaciotemporales. La gran variedad que ofrece JAAD le ha permitido ser ampliamente utilizado en el desarrollo tanto de modelos orientados a la predicción temprana sobre el peatón como en el análisis de situaciones complejas entre múltiples agentes.

A continuación, se muestran las estadísticas generales de JAAD [31]:

- **Número total de fotogramas:** 82032
- **Número total de fotogramas anotados:** 82032
- **Número de peatones con anotaciones de comportamiento:** 686
- **Número total de peatones:** 2786
- **Número de *bounding boxes* sobre peatones:** 378643
- **Longitud media del recorrido de un peatón:** 121 fotogramas

Aquí se muestran el conteo de peatones basado en su comportamiento:

- **Número de peatones que cruzan la calle:** 495
- **Número de peatones que no cruzan la calle:** 191

Para este trabajo, no se van a usar directamente los datos de JAAD, sino una adaptación donde los datos son preprocesados y enriquecidos para facilitar la modelización del comportamiento del peatón para lograr que sea más estructurable y explicable (véase Figura 4.1).

Esta adaptación de los datos se basa en características de alto nivel extraídas gracias a la información contextual y visual de los vídeos, como la posición del peatón respecto al vehículo, la orientación, o la acción en un instante determinado. Estas características permiten transformar los vídeos en datos analizables mediante modelos GNN.

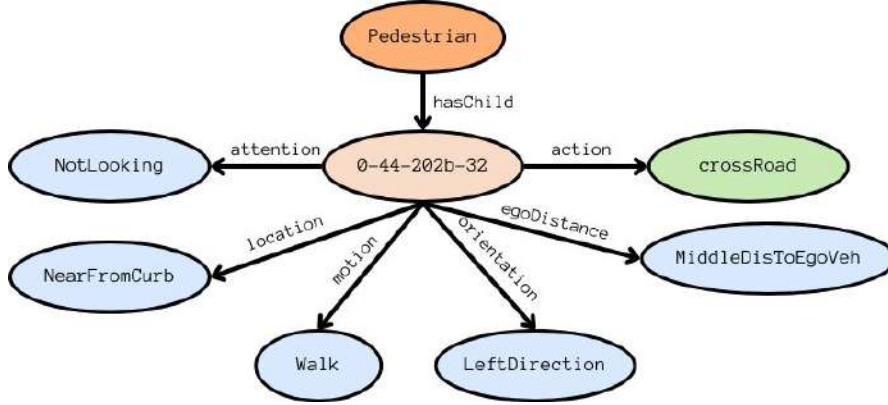


Figura 4.1: Grafo adaptado con los datos de JAAD [2]

Además, a partir de esta adaptación de JAAD, se distinguen dos tipos de representaciones de los datos, dependiendo del nivel de abstracción: datasets numéricos y datasets lingüísticos.

Los conjuntos de datos numéricos representan cada muestra mediante valores cuantitativos y categóricos aprovechables por modelos de ML. Las *features* del peatón se expresan mediante valores numéricos o etiquetas discretas, preservando así la estructura temporal a nivel frame. Este formato de datos resulta adecuado en arquitecturas como las GNN debido a que las relaciones entre entidades pueden modelarse de forma explícita.

Por otro lado, los datos lingüísticos elevan el nivel de abstracción, ya que representan información mediante descripciones semánticas y reglas simbólicas. En este caso, las mismas *features* se expresan mediante conceptos lingüísticos y combinaciones de reglas que describen el comportamiento del peatón. Este nuevo enfoque facilita la interpretación del comportamiento, permitiendo conectar datos con sistemas basados en conocimiento y razonamiento explicable.

Ambas representaciones de datos son adaptaciones complementarias del dataset JAAD, ya que mantienen el contexto del comportamiento del peatón, permitiendo tratar el problema mediante enfoques numéricos y simbólicos. En consecuencia, el uso de una versión adaptada de los datos originales proporciona un dataset alineado con el objetivo del trabajo, ya que permite estudiar el comportamiento del peatón de manera estructurada, manteniendo el contexto temporal del dataset JAAD.

4.4.1. Sumario

En el grafo adaptado, las características utilizadas son las siguientes:

- **video:** ID del vídeo utilizado.
- **frame:** Número exacto del frame.

- **person**: ID del peatón.
- **attention**: Atención visual del peatón.
- **orientation**: Dirección del peatón en grados.
- **proximity**: Proximidad entre peatón y cruce.
- **zebra_cross**: Indica si hay paso de cebra o no.
- **distance**: Distancia entre peatón y vehículo.
- **action**: Acción física del peatón.
- **cross**: Indica si el peatón está cruzando o no.

4.5. KNIME

Konstanz Information Miner (KNIME) es una plataforma de análisis de código abierto orientada al análisis de datos, gracias a un diseño de flujos visuales para tareas de procesamiento, análisis y minería de datos. La gran ventaja que ofrece KNIME es la posibilidad de crear *pipelines* de forma modular y reproducible, sin necesidad de codificar todo el procesamiento de datos, por lo que facilita la validación de datos [32].

Para este trabajo, KNIME ha servido como herramienta de apoyo para analizar el conjunto de datos utilizados, permitiendo evaluar la calidad de los datos, posibles inconsistencias y la presencia de ruido. Gracias a su uso, se puede complementar un enfoque técnico basado en DL con un análisis previo más interpretativo, facilitando la toma de decisiones sobre el preprocesamiento de los datos. De este modo, KNIME es una plataforma clave para garantizar la fiabilidad de los datos empleados.

Capítulo 5

Diseño del sistema y metodología

En este capítulo se detalla el diseño del sistema utilizando las herramientas y datos comentados en el anterior capítulo.

5.1. Diseño del grafo y representación de datos

El diseño del grafo constituye un elemento base del sistema, ya que es donde se determina las estructuras y relaciones de los datos de entrada antes de ser procesados por los modelos. Los datos procedentes de la adaptación de JAAD son transformados en distintos tipos de grafos, yendo desde grafos básicos hasta grafos con relaciones espaciotemporales.

Cada nodo del grafo representa a un peatón en un instante temporal concreto, y está caracterizado por un vector de atributos con las *features* del peatón. Estas características incluyen la atención visual, la proximidad al cruce, la orientación, la distancia al vehículo y la acción realizada. Todas estas variables son codificadas y combinadas en un vector de tamaño fijo, generando una representación homogénea de los nodos independientemente del número de peatones en la escena.

5.1.1. Grafos espaciales

En cuanto a la estructura espacial del grafo, se han contemplado tres diseños en el sistema. Estos diseños son configurables mediante una variable global, con el objetivo de analizar como diferentes representaciones afectan al aprendizaje del modelo. El primer diseño (véase Figura 5.1a) tiene una estructura en forma de estrella, donde se establece conexiones entre un nodo central peatón y el resto de nodos relevantes, priorizando así las relaciones directas entre el peatón y su entorno inmediato.

El segundo diseño (véase Figura 5.1b) modifica la estructura inicial con una conectividad de aristas alternativa, donde se prioriza una relación más realista y causal entre las variables observadas, en concreto, la relación entre la distancia y la acción.

Por último, el tercer grafo (véase Figura 5.1c) combina ambos enfoques, integrando las aristas de las dos primeras estructuras de diseño para capturar una influencia directa e indirecta sobre la acción, modelando interacciones simultáneas entre factores.

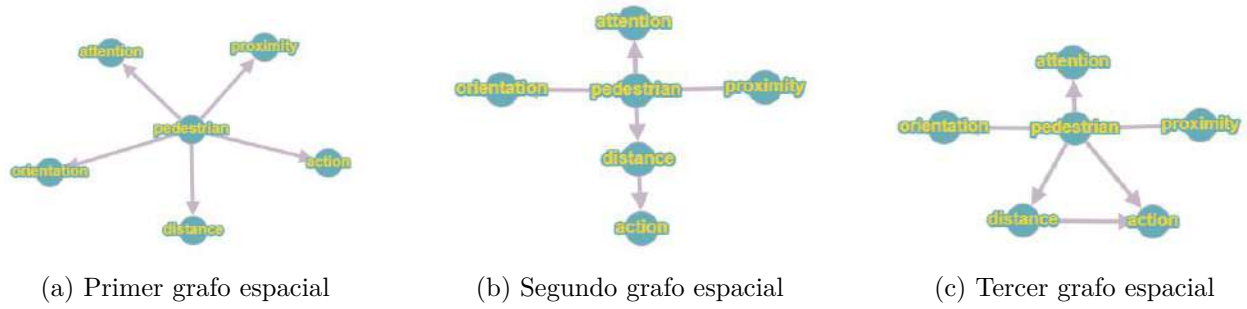


Figura 5.1: Representación conceptual de los tres grafos espaciales

5.1.2. Grafos temporales

Además de la estructura espacial, el sistema también permite incorporar dependencias temporales gracias a una conexión entre distintos frames consecutivos utilizando aristas temporales que conectan nodos en instantes temporales consecutivos, todo esto controlado con una variable global. La primera prueba espaciotemporal sirve de base, ya que solo se conecta el peatón entre frames consecutivos, definiendo una relación básica que será aumentada en las siguientes estructuras (véase Figura 5.2).

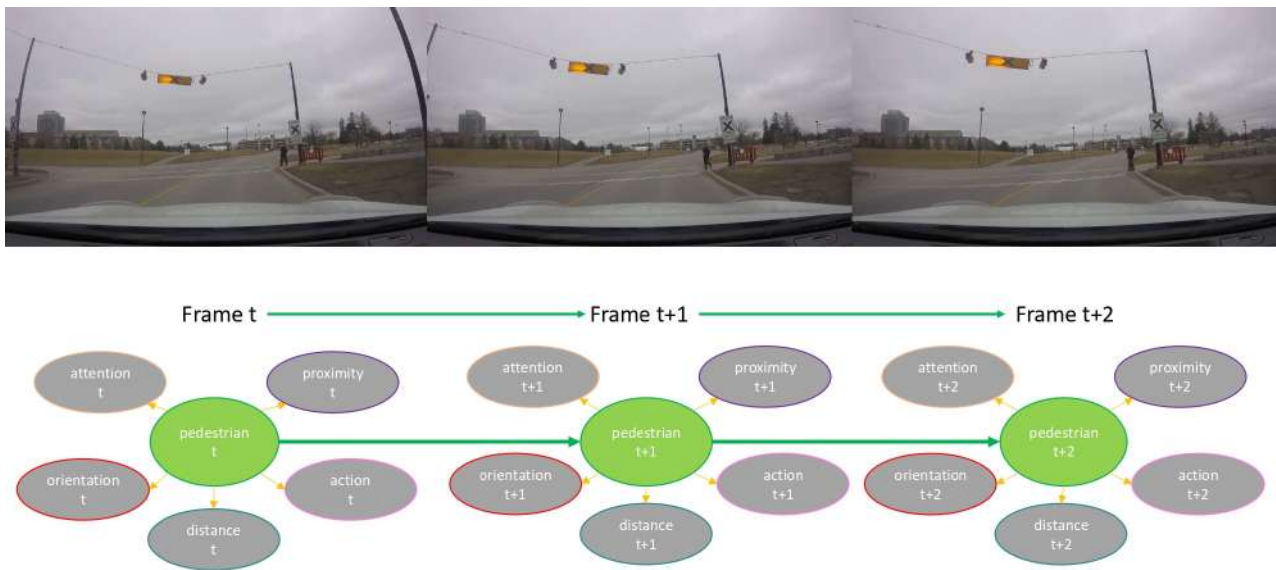


Figura 5.2: Representación conceptual del primer grafo espaciotemporal

Utilizando esta estructura base, se puede definir otro grafo espaciotemporal completo, donde se conectan todos los nodos equivalentes entre frames y todas las relaciones espaciales dentro de cada frame (véase Figura 5.3).

5.1.3. Sliding Window

Por último, se introduce el concepto de ventanas deslizantes (*sliding windows*), donde se utiliza una ventana de frames consecutivos para capturar la evolución temporal local de los datos mediante un promedio vectorial, generando un único grafo representativo de la ventana, sin tener que conectar todo el grafo globalmente (véase Figura 5.4).

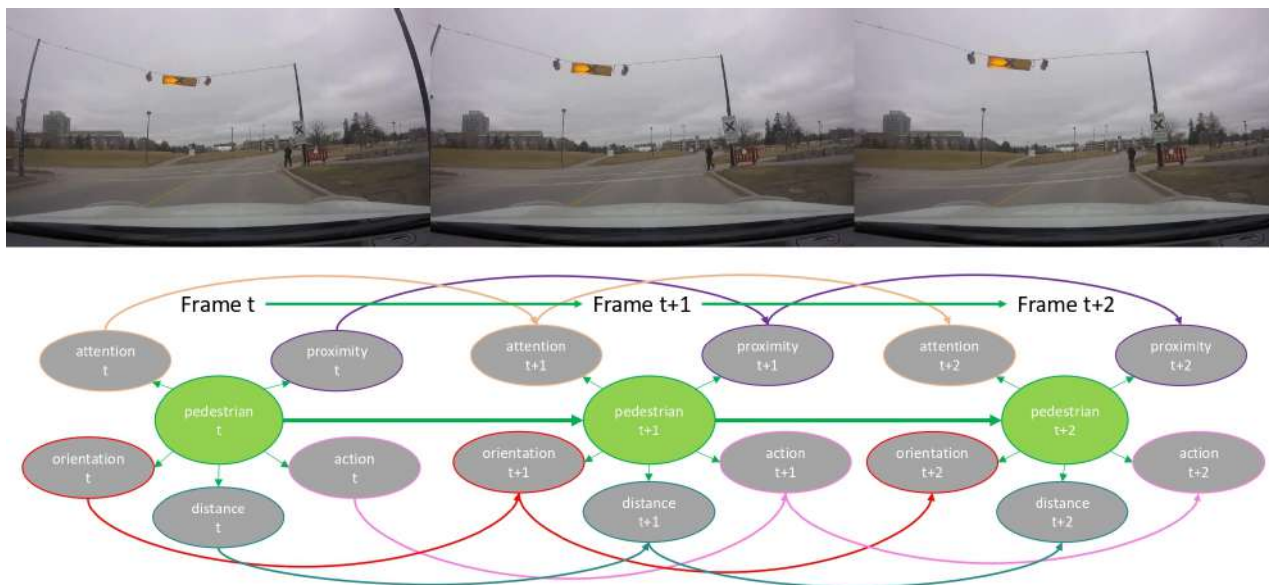


Figura 5.3: Representación conceptual del segundo grafo espaciotemporal

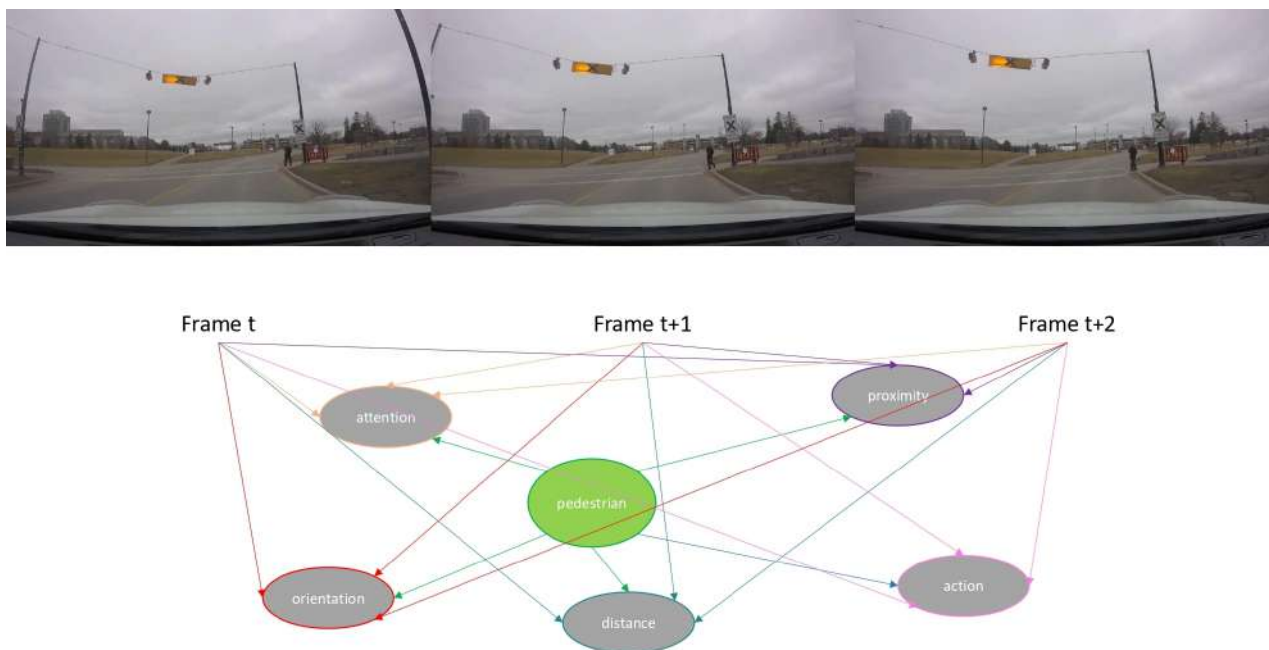


Figura 5.4: Representación conceptual del tercer grafo espaciotemporal

5.1.4. Caso MultiPeaton

El sistema también permite trabajar tanto con grafos de un solo peatón como con grafos multi-peatón por escena. En este último modo, cada grafo representa una escena completa de tráfico. Dentro de cada frame, se conectan los peatones mediante aristas espaciales completas, mientras que a nivel escena se conectan mediante aristas temporales, capturando las interacciones entre peatones y su evolución temporal, lo que ayuda a proporcionar una visión global de la escena. La etiqueta de cruce tiene dos tipos de definiciones, ya sea en función de la etiqueta mayoritaria, o en función de que al menos un peatón esté cruzando. Este enfoque permite enriquecer la representación del contexto, permitiendo así que el modelo capture las relaciones sociales implícitas y patrones colectivos que no

pueden ser representados cuando se estudia a un peatón de forma aislada.

El diseño del grafo y la representación de los datos sirven como base representativa para el análisis del comportamiento del peatón, permitiendo estudiar el impacto tanto de estructuras espaciales como temporales sobre el rendimiento de modelos GNN.

5.2. Pipeline experimental

El *pipeline* experimental define la secuencia completa desde la preparación de los datos hasta la evaluación final de los modelos, siendo un proceso sistemático, reproducible y comparable con distintas configuraciones. Este diseño permite un análisis controlado del impacto de diferentes representaciones de grafo, modelos GNN e hiperparámetros utilizados.

Inicialmente, los datos adaptados de JAAD son cargados y preprocesados. En este proceso, se seleccionan las *features* de interés, normalizando valores continuos y codificando variables categóricas, generando así los vectores de atributos asociados a cada nodo. A continuación, se construyen los grafos en función de la configuración seleccionada mediante las variables globales del sistema, pudiendo variar tanto la estructura espacial como las relaciones temporales.

Una vez que ya han sido generados los grafos, el conjunto de datos se divide en dos, el conjunto de entrenamiento y el conjunto de test, evaluando el comportamiento del modelo durante el aprendizaje y su capacidad de generalización sobre datos no vistos. Este entrenamiento se realiza de manera supervisada, con una función de pérdida de clasificación optimizada y con un ajuste de los pesos del modelo mediante el descenso de gradiente.

Durante el entrenamiento, se registran la pérdida y exactitud del modelo tanto en entrenamiento como en validación, ayudando a detectar sobreajuste. Una vez acabado el entrenamiento, el modelo se evalúa sobre el conjunto de test, obteniendo una estimación final del rendimiento. Todo este proceso experimental se puede repetir bajo distintas configuraciones de grafos y modelos, comparando consistencias entre enfoques y la robustez de los resultados obtenidos. Este *pipeline* experimental sirve como base sólida para la evaluación del impacto en las decisiones de diseño, lo que facilita el análisis comparativo entre representaciones.

5.3. Modelos GNN utilizados

Para este trabajo se han utilizado tres distintas arquitecturas GNN con el objetivo de analizar cómo diferentes mecanismos de propagación de información afectan al comportamiento del peatón. Esta selección de modelos busca un equilibrio entre simplicidad y capacidad expresiva, permitiendo así una comparación justa entre modelos.

Los tres modelos seleccionados son Graph Convolutional Networks (GCN), Graph Attention Networks (GAT) y GraphConv. Estos modelos se diferencian en la forma en que agregan información de los nodos vecinos. Gracias a estas diferencias, se puede estudiar el impacto de distintos modelos sobre grafos espaciales y espaciotemporales.

5.3.1. Graph Convolutional Networks

Las Graph Convolutional Networks (GCN) se basan en la propagación de información mediante una normalización de la matriz de adyacencia, por lo que cada nodo puede actualizar su representación combinando las características propias con la de los vecinos inmediatos. Este proceso se formaliza mediante la regla de propagación definida para cada capa de la red (véase Ecuación (5.1)), que permite integrar la información estructural del grafo con los atributos de los nodos y se expresa de la siguiente manera [33]:

$$H^{(l+1)} = \sigma \left(\tilde{D}^{-\frac{1}{2}} \tilde{A} \tilde{D}^{-\frac{1}{2}} H^{(l)} W^{(l)} \right) \quad (5.1)$$

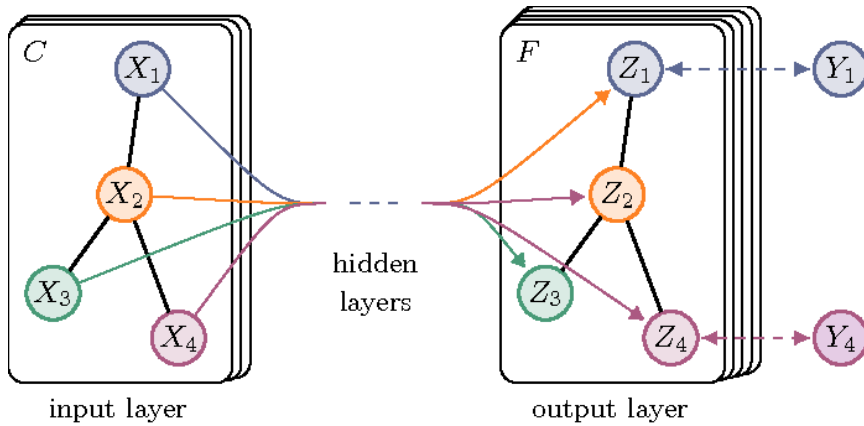


Figura 5.5: Esquema de las multi capas en GCN [33]

Una característica de las GCN es la contribución homogénea de los nodos vecinos en el proceso de agregación, dando lugar a un modelo sencillo, estable y eficiente computacionalmente hablando (véase Figura 5.5). Las GCN van a servir como modelo base, siendo referencia para evaluar arquitecturas más complejas y mecanismos de agregación más expresivos.

5.3.2. Graph Attention Networks

Las Graph Attention Networks (GAT) surgen con el objetivo de mejorar la capacidad para modelar grafos heterogéneos gracias a los mecanismos de atención. En vez de agregar el mismo peso a los nodos vecinos (como hacen las GCN), las GAT aprenden un coeficiente de atención el cual permite ponderar de forma desigual a los nodos adyacentes [34] (véase Figura 5.6).

Este avance resulta interesante en contextos con mayor complejidad, donde no toda la información en las relaciones resultan de igual relevancia para la predicción. Las GAT ofrecen mayor precisión a la hora de modelar la influencia relativa entre variables, proporcionando mayor capacidad para capturar patrones relevantes en situaciones dinámicas.

5.3.3. GraphConv

El modelo *GraphConv* supone una formulación más general de las GNN, siendo ampliamente utilizado en PyG. Este tipo de convolución permite separar explícitamente la contribución del nodo

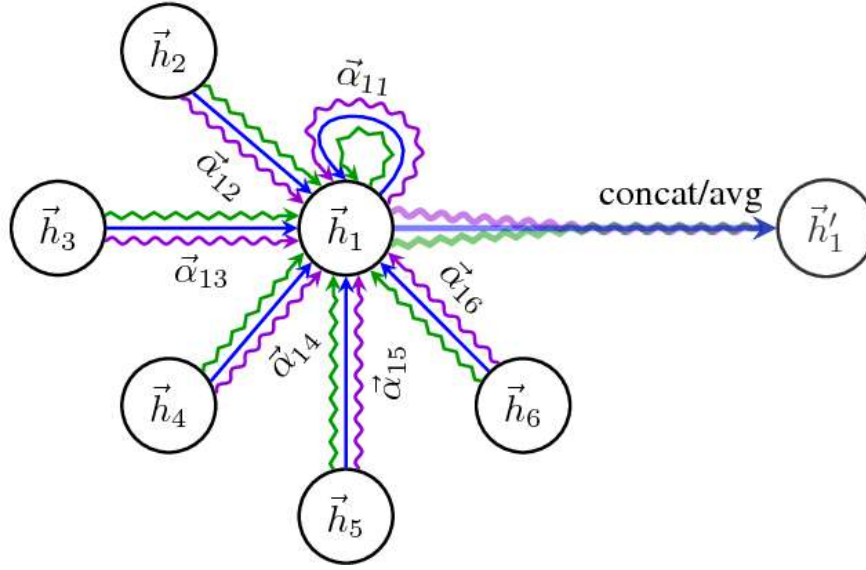


Figura 5.6: Ilustración de las múltiples cabezas en GAT [34]

central de la agregación de información de los vecinos, ofreciendo mayor flexibilidad en el aprendizaje. A diferencia de otras arquitecturas, *GraphConv* diferencia a la hora de modelar entre la influencia propia del nodo y la de su entorno, facilitando la adaptación a distintas estructuras.

Esta nueva formulación favorece a las relaciones locales del grafo, capturando patrones estructurales relevantes sin imponer restricciones sobre la forma de agregar. Este tipo de arquitecturas más flexibles presentan una mayor capacidad para adaptarse a grafos con estructuras complejas, controlando a su vez la complejidad computacional [35]. Trabajar con este modelo resulta interesante ya que las relaciones entre nodos pueden variar significativamente entre configuraciones espaciales e instantes temporales.

5.4. Configuración experimental e hiperparámetros

La configuración experimental define los parámetros utilizados tanto para la fase de entrenamiento como para la de evaluación, asegurando un entorno controlado y reproducible para comparar diferentes arquitecturas y modelos. Para realizar cualquier experimento, se ha utilizado la misma configuración base, modificando únicamente los elementos relacionados con el grafo o el modelo GNN (véase Figura 5.7).

En primer lugar, el conjunto de datos se divide en tres subconjuntos, el de entrenamiento, el de validación y el de test. De todos los grafos generados, el 80 % se destina al entrenamiento, y el 20 % restante se utiliza como conjunto de validación para monitorizar el aprendizaje. El conjunto de test se encuentra separado de estos dos, ya que se utiliza para la evaluación final del modelo, y esta estimación debe ser objetiva para el proceso de generalización.

El rendimiento de los modelos GNN seleccionados depende en gran medida de los hiperparámetros estructurales (véase Tabla 5.1). La configuración experimental define los valores, así como los criterios comunes para realizar una comparativa igualitaria entre arquitecturas. Los valores se basan

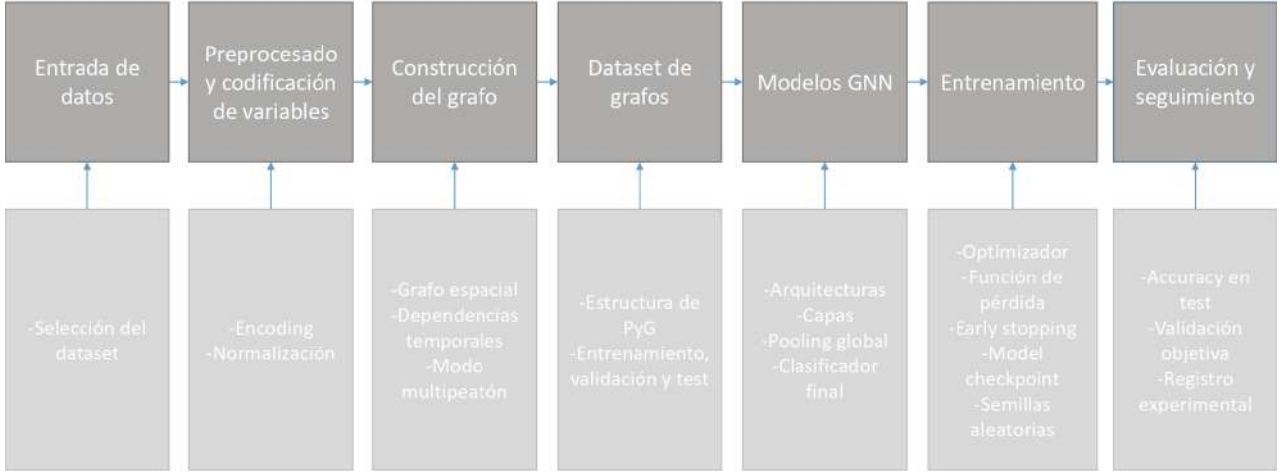


Figura 5.7: Ilustración del *pipeline* utilizado

en experimentos preliminares y diversos análisis de estabilidad del entrenamiento, con el fin de buscar un equilibrio entre aprendizaje, robustez y generalización.

En líneas generales, se emplean arquitecturas con entre dos a cuatro capas convolucionales, utilizando dimensiones ocultas de 64 o 128 unidades, y la función de activación Unidad Lineal Rectificada (ReLU). Para reducir el sobreajuste, se aplica *dropout* en las capas GNN y en el clasificador final. El proceso de optimización se realiza gracias a la tasa de aprendizaje del optimizador *AdamW*. La función de pérdida *nn.BCEWithLogitsLoss* combina una capa *Sigmoid* y *Binary Cross Entropy* (entropía cruzada binaria) en una sola clase. Esta configuración garantiza una convergencia estable del entrenamiento y evaluación en distintos escenarios experimentales.

5.5. Seguimiento experimental con Wandb

Para poder realizar un seguimiento experimental sistemático, garantizando una trazabilidad de resultados, se ha utilizado la plataforma *Weights & Biases* (Wandb) (véase Sección 4.2) para la monitorización y registro de experimentos. Durante el entrenamiento, se ha empleado para almacenar métricas de pérdida, además de medidas de rendimiento. Gracias a este seguimiento, se puede hacer un análisis visual y detallado sobre la convergencia de los modelos, comparando éstos en un escenario experimental equivalente.

Además, la ejecución con Wandb garantiza reproducibilidad, ya que las ejecuciones se asocian al conjunto de hiperparámetros y configuraciones utilizados, evitando ambigüedad en las comparaciones. Esta funcionalidad resulta de gran relevancia debido a la gran variedad experimental, ya sea por los tipos de grafos, las configuraciones temporales y las arquitecturas GNN. En conjunto, esta herramienta proporciona un control y organización adicional al proceso de pruebas, siendo de gran ayuda para hacer una evaluación más transparente con los resultados y el análisis comparativo de éstos.

Para las gráficas utilizadas en este trabajo, se van a visualizar los siguientes parámetros (véase Figura 5.8):

Parámetro	Modelos	Descripción	Ajuste
c_hidden	GCN, GAT y GraphConv	Número de neuronas por capa. Controla la capacidad del modelo	Si se aumenta, mejora la representación, pero puede causar sobreajuste. Los valores típicos suelen ser 32, 64 o 128
num_layers	GCN, GAT y GraphConv	Número de capas GNN apiladas. Controla la profundidad del mensaje propagado entre nodos	El valor óptimo suele ser 2 o 3, ya que más capas puede provocar <i>over smoothing</i>
dp_rate	GCN, GAT y GraphConv	Tasa de dropout dentro de las capas GNN. Sirve para reducir el sobreajuste	Lo normal es entre 0.1 y 0.5. Cuanto mayor es el dropout, mayor es la regularización
dp_rate_linear	GCN, GAT y GraphConv	Dropout antes de la capa lineal final. Controla la regularización del clasificador	Un valor más alto que dp_rate, a partir de 0.5
heads	GAT	Número de cabezas de atención en cada capa, permitiendo atender a diferentes subconjuntos de vecinos	Suele ser 4 u 8; a mayor número, más expresividad, pero mayor coste computacional
concats	GAT	Indica si se concatenan las salidas de las cabezas de atención	True en capas ocultas y False en la última capa
aggr	GCN y GraphConv	Método de agregación de mensajes	<i>mean</i> suele ser estable, <i>add</i> puede mejorar señales débiles
lr	GCN, GAT y GraphConv	Tasa de aprendizaje del optimizador AdamW	El valor habitual es 1×10^{-3} , pero puede ser conveniente reducirlo si hay oscilaciones
batch_size	GCN, GAT y GraphConv	Número de grafos procesados por batch	Depende del tamaño de los grafos. Para grafos pequeños, con 8 es suficiente
max_epochs	GCN, GAT y GraphConv	Número de épocas de entrenamiento	Suele estar entre 50 y 200, dependiendo de la convergencia
global_mean_pool	GCN, GAT y GraphConv	Tipo de pooling global (mean, max, sum) para obtener la representación del grafo	Cambiarlo puede mejorar la estabilidad

Tabla 5.1: Hiperparámetros configurables de los modelos GNN

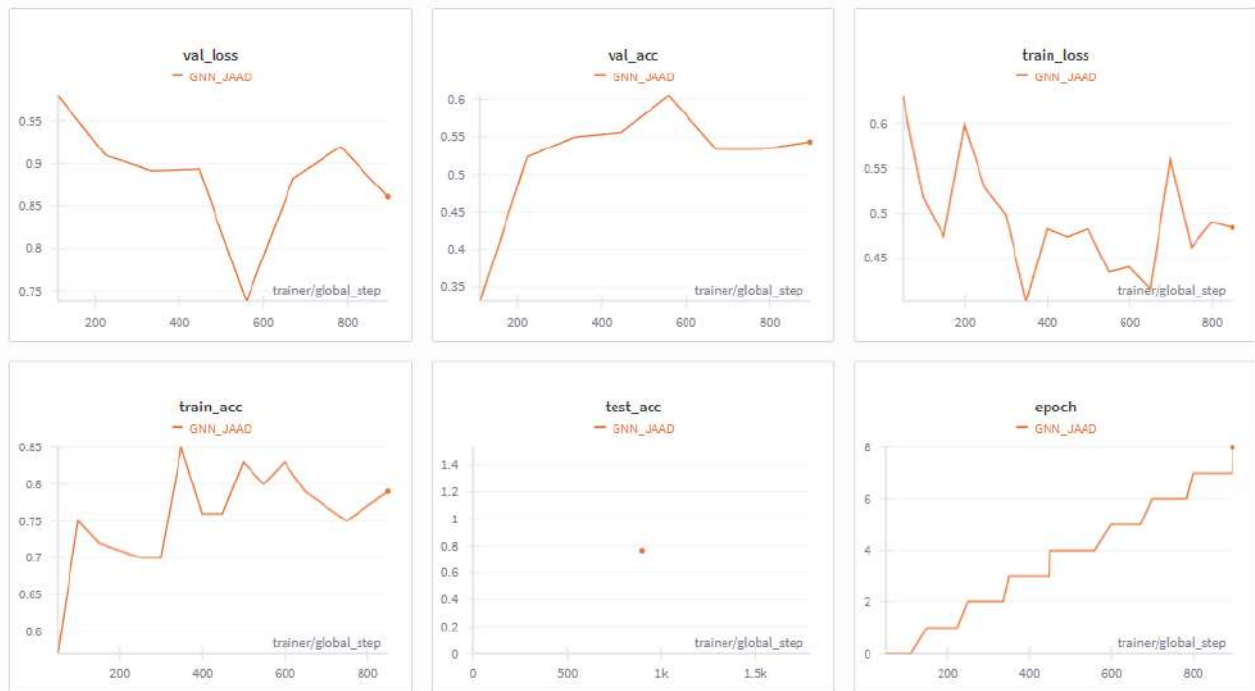


Figura 5.8: Ejemplo de resultados en Wandb

1. **val_loss**: Pérdida en el conjunto de validación, evaluando el desempeño del modelo en datos no vistos durante el entrenamiento.
2. **val_acc**: Exactitud en el conjunto de validación, indicando la precisión en las predicciones sobre los ejemplos de validación.
3. **train_loss**: Pérdida en el conjunto de entrenamiento, calculando el error del modelo sobre datos de entrenamiento.
4. **train_acc**: Exactitud en el conjunto de entrenamiento, mostrando la precisión en las predicciones sobre los ejemplos de entrenamiento.
5. **test_acc**: Exactitud en el conjunto de test, valorando la capacidad de generalización del modelo después del entrenamiento.
6. **epoch**: Número de épocas, es decir, el número de veces que ha pasado el modelo por el conjunto de datos de entrenamiento.

5.6. Seguimiento en GitHub

Tanto el código utilizado como la gestión del proyecto ha sido gestionado con GitHub, siendo el eje central para el control de versiones, planificación de tareas y seguimiento del trabajo. Gracias al uso de esta herramienta, se ha podido tener un desarrollo ordenado y reproducible habitual en prácticas de entornos profesionales.

En primer lugar, todo el código ha sido versionado mediante Git, estando alojado en un repositorio público [36]. A lo largo del desarrollo del proyecto se han realizado numerosos *commits* donde se

refleja la evolución del sistema, empezando por las primeras implementaciones de los grafos espaciales hasta las últimas extensiones del modelado multipeatón. Gracias a este historial de cambios, se puede identificar claramente las distintas fases del trabajo, además de poder revertir modificaciones concretas si fuese necesario (véase Figura 5.9).



Figura 5.9: Ejemplo de *commits* en GitHub

Además del control de versiones, se ha utilizado la herramienta de gestión de proyectos propia de GitHub, llamada GitHub Projects. Gracias a un tablero de planificación, se han organizado las tareas en sus distintas fases. Este enfoque permite descomponer el trabajo en objetivos manejables, facilitando así el seguimiento del estado de cada tarea, pero manteniendo una visión global del proyecto (véase Figura 5.10).

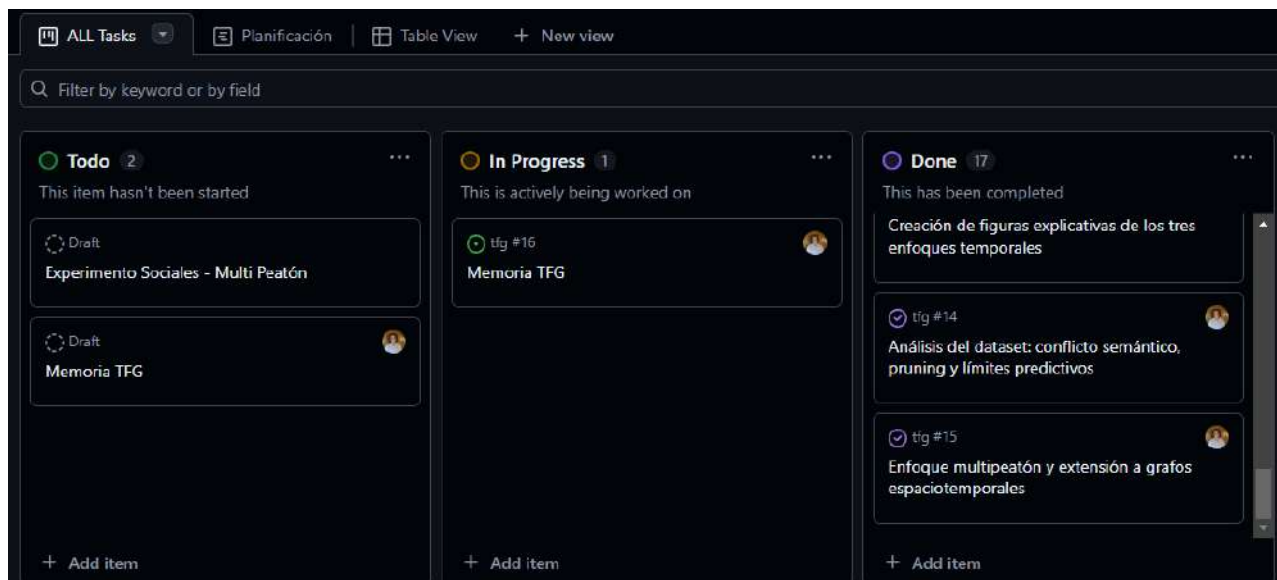
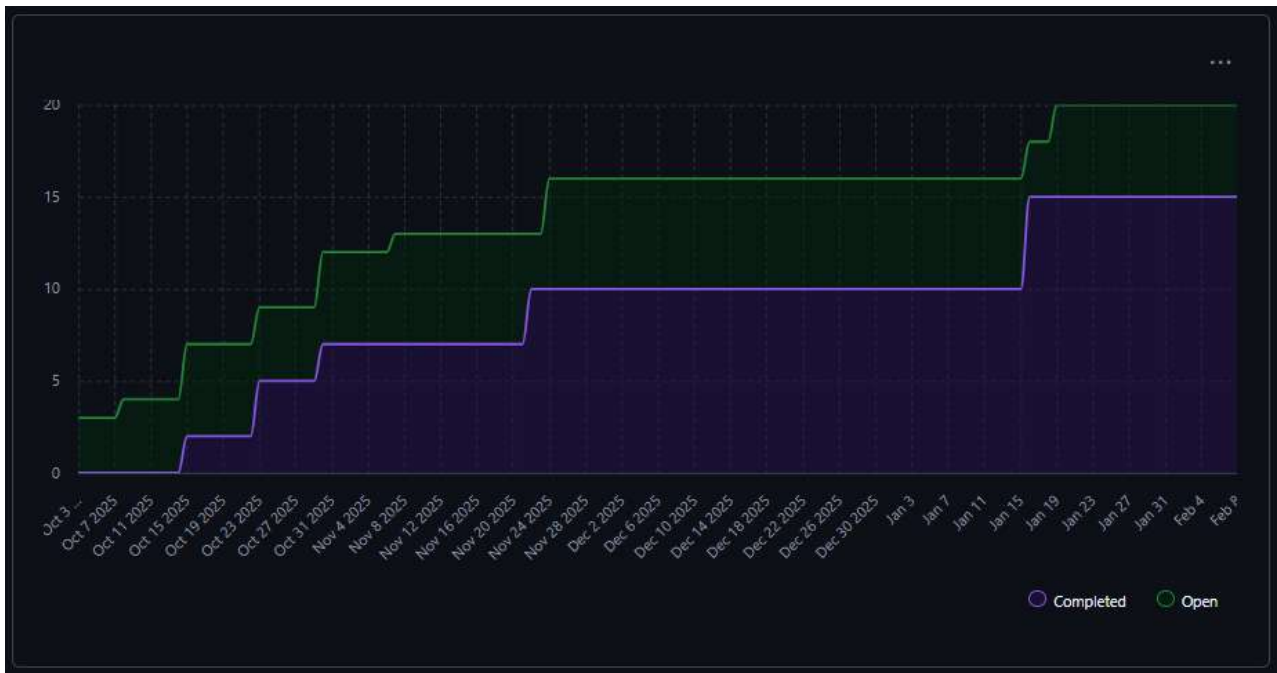


Figura 5.10: Tablero de planificación en GitHub

Por otra parte, también se ha utilizado la sección *Insights*, donde se ha analizado la actividad del repositorio, incluyendo el número de *commits* y la distribución del trabajo. Esta información resulta de gran utilidad para evidenciar la constancia en el desarrollo del proceso (véase Figura 5.11).

Figura 5.11: *Insights* en GitHub

En conjunto, tanto el uso de GitHub como de sus propias herramientas ha permitido un desarrollo estructurado del sistema propuesto, además de ser una gran ayuda para adquirir experiencia práctica en metodologías de trabajo colaborativas, siendo un hecho fundamental en proyectos reales de investigación.

Capítulo 6

Pruebas y resultados

En este capítulo se presentan las pruebas realizadas con sus correspondientes resultados. Para estas pruebas se han utilizado tanto el dataset MUTAG como JAAD, además de los tres modelos GNN ya comentados. Para las pruebas, se calcula el *accuracy* con el objetivo de comparar como funcionan las GNN.

6.1. Resultados sobre MUTAG

El banco de datos MUTAG ha servido como fuente inicial para validar el correcto funcionamiento del *pipeline* experimental, teniendo una primera toma de contacto con el comportamiento de los modelos GNN utilizados en un entorno controlado. Al ser un conjunto de datos clásico en la clasificación de grafos, ha permitido contrastar las pruebas con referencias ampliamente conocidas.

El modelo GCN presenta un número reducido de parámetros (aproximadamente 4.8K) debido a su sencilla arquitectura, utilizando una matriz de pesos por capa, siendo un buen modelo base para realizar las comparaciones. Los resultados obtenidos con distintas configuraciones (véase Tabla 6.1) muestran un *accuracy* entre 0.68 y 0.73, confirmando que este modelo es adecuado como referencia inicial, pero puede verse limitado en escenarios más expresivos (véase Figura 6.1).

Configuración	c_hidden	num_layers	dp_rate	lr	batch_size
Primera configuración	64	2	0.1	1×10^{-3}	8
Segunda configuración	128	3	0.3	1×10^{-3}	8
Tercera configuración	64	2	0.5	5×10^{-4}	16
Cuarta configuración	64	4	0.3	1×10^{-3}	8

Tabla 6.1: Configuraciones para GCN

Por otro lado, GAT incrementa notablemente el número de parámetros entrenables (en torno a 19.3K) gracias a la incorporación de los mecanismos de atención, ya que utiliza un vector de atención donde se calcula la importancia de los vecinos. Los resultados sobre MUTAG con distintas configuraciones (véase Tabla 6.2) se encuentran entre 0.63 y 0.71, donde se puede observar una tendencia al sobreajuste debido al reducido tamaño del dataset. La expresividad característica de GAT no siempre significa una mejora en conjuntos de datos pequeños (véase Figura 6.2).

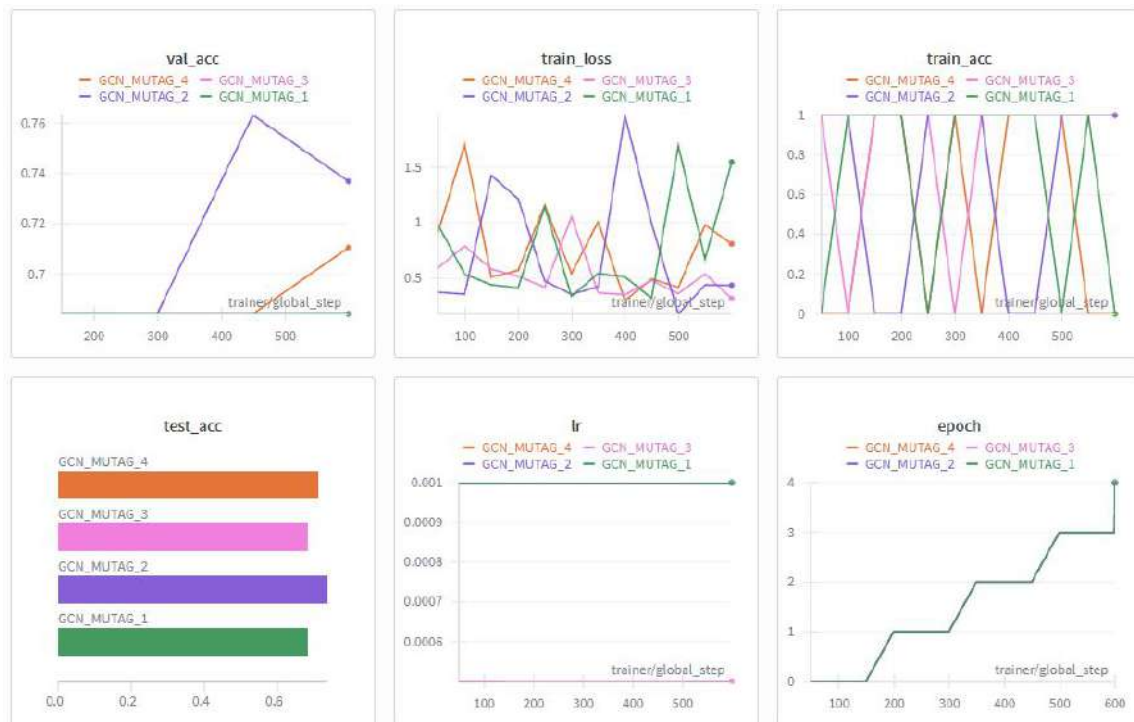


Figura 6.1: Resultados sobre MUTAG usando GCN

Configuración	c_hidden	num_layers	dp_rate	lr	batch_size	heads	concat
Primera configuración	64	2	0.2	1×10^{-3}	8	4	True
Segunda configuración	128	3	0.3	1×10^{-3}	8	8	True
Tercera configuración	64	2	0.5	5×10^{-4}	16	4	False
Cuarta configuración	64	4	0.4	1×10^{-3}	8	6	True

Tabla 6.2: Configuraciones para GAT

Por último, GraphConv combina simplicidad y capacidad expresiva, utilizando un número de parámetros entrenables intermedio (alrededor de 9.3K), aportando flexibilidad en el aprendizaje, ya que tiene pesos entrenables tanto para el nodo central como para los nodos vecinos. Los experimentos con MUTAG con distintas configuraciones (véase Tabla 6.3) muestran unos resultados entre 0.68 y 0.76, siendo el modelo que alcanza el resultado más alto de los tres, mostrando también la mejor capacidad de generalización (véase Figura 6.3).

Configuración	c_hidden	num_layers	dp_rate	lr	batch_size	dp_rate_linear
Primera configuración	64	2	0.1	1×10^{-3}	8	0.5
Segunda configuración	128	3	0.3	1×10^{-3}	8	0.5
Tercera configuración	64	2	0.5	5×10^{-4}	16	0.6
Cuarta configuración	64	4	0.3	1×10^{-3}	8	0.5

Tabla 6.3: Configuraciones para GraphConv

Esta comparación inicial (véase Tabla 6.4) pone de manifiesto las diferencias entre los tres modelos utilizados. Mientras que GCN destaca por su facilidad y simplicidad, GAT introduce una mayor capacidad expresiva, incrementando así la complejidad y el riesgo de sobreajuste, y GraphConv logra un balance entre ambos, obteniendo el mejor resultado global de los experimentos iniciales. Con estos

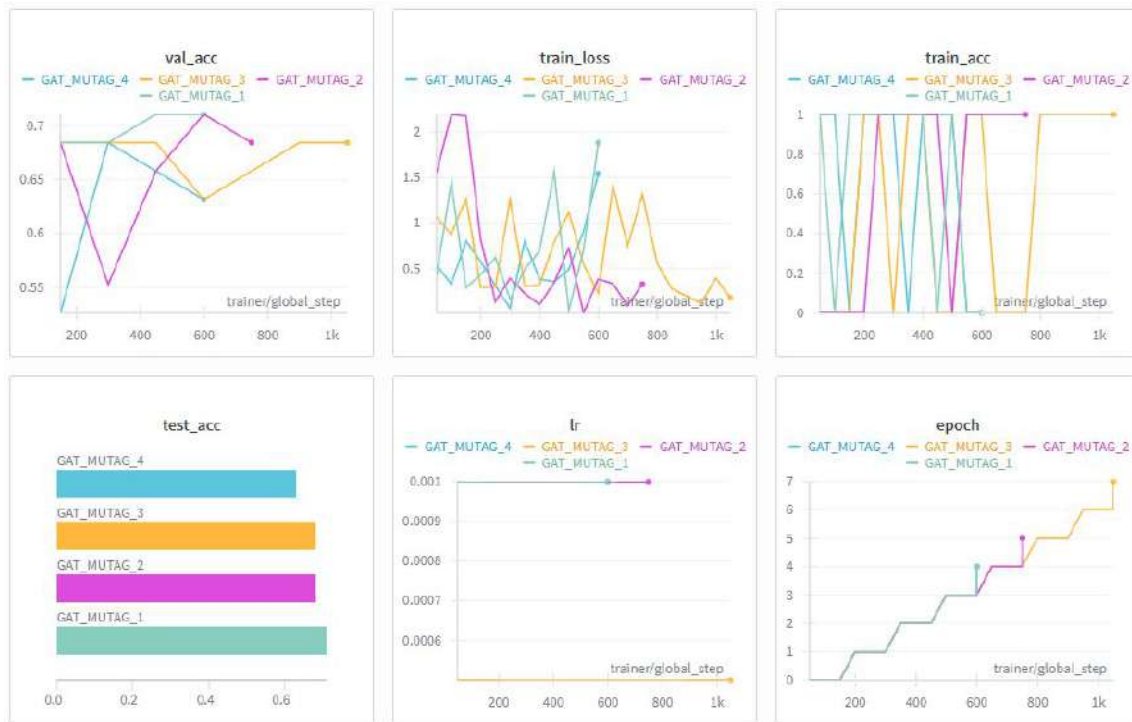


Figura 6.2: Resultados sobre MUTAG usando GAT

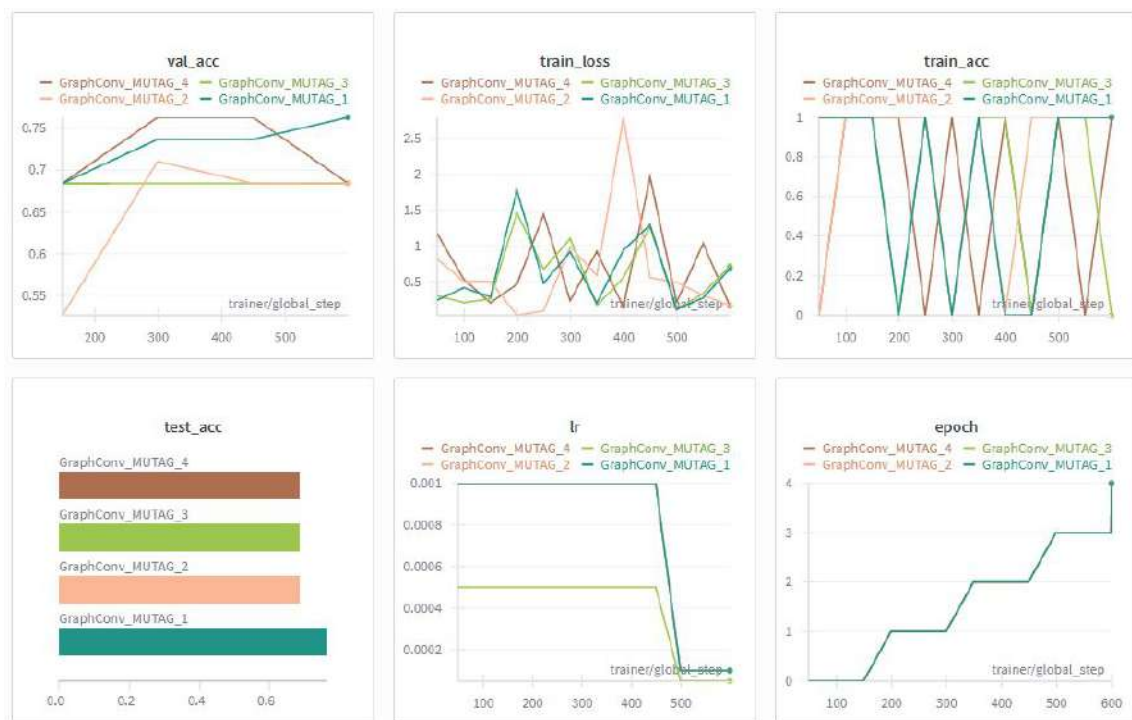


Figura 6.3: Resultados sobre MUTAG usando GraphConv

resultados se puede validar el *pipeline* empleado, justificando así la selección de los modelos GNN para escenarios más complejos derivados de los datos de JAAD.

Modelo	Configuración	test_acc
GCN	Primera configuración	0.6842
GCN	Segunda configuración	0.7368
GCN	Tercera configuración	0.6842
GCN	Cuarta configuración	0.7105
GAT	Primera configuración	0.7105
GAT	Segunda configuración	0.6842
GAT	Tercera configuración	0.6842
GAT	Cuarta configuración	0.6315
GraphConv	Primera configuración	0.7631
GraphConv	Segunda configuración	0.6842
GraphConv	Tercera configuración	0.6842
GraphConv	Cuarta configuración	0.6842

Tabla 6.4: Comparativa de resultados con el dataset MUTAG

6.2. Resultados sobre JAAD

El escenario principal de evaluación sobre el sistema propuesto lo constituyen los resultados obtenidos sobre los datos provenientes de JAAD. A diferencia de otros datasets como MUTAG, JAAD se basa en un entorno realista y dinámico basado en el comportamiento humano, caracterizado por dependencias temporales, la presencia de múltiples agentes y una gran influencia del contexto. Gracias a todo esto, JAAD permite evaluar la capacidad de generalización de los modelos GNN en un contexto real.

Estos experimentos se han realizado sobre las tres arquitecturas definidas (GCN, GAT y GraphConv), utilizando los tres grafos espaciales así como las dos variantes de datos, tanto la numérica como la lingüística. Mediante esta elección de pruebas, se puede realizar un análisis de forma sistemática sobre el impacto de cada decisión de diseño sobre el rendimiento.

A partir de esta configuración experimental, los resultados se van a analizar en función del diseño del grafo espacial y en función del formato de los datos utilizados.

6.2.1. Comparativa según el diseño del grafo

Esta comparativa se basa en analizar el comportamiento de los modelos al variar la estructura del grafo, manteniendo constante la arquitectura GNN y el tipo de datos. El primer grafo espacial (véase Figura 5.1a), caracterizado por una estructura en estrella y simple, sirve como base para el aprendizaje, aunque limita la capacidad de los modelos cuando aumenta la complejidad predictiva. En este grafo, el modelo GCN es el más estable pero también el que presenta menor capacidad de mejora (véase Figura 6.4).

En cuanto al segundo grafo espacial (véase Figura 5.1b), se introduce una conectividad alternativa al primero, priorizando la relación entre distancia y acción del peatón. Esta modificación ha significado un ligero empeoramiento general del rendimiento, sobretodo en arquitecturas más expresivas. Los modelos muestran mayor sensibilidad frente a este grafo debido a las relaciones introducidas (véase Figura 6.5).



Figura 6.4: Resultados utilizando el primer grafo

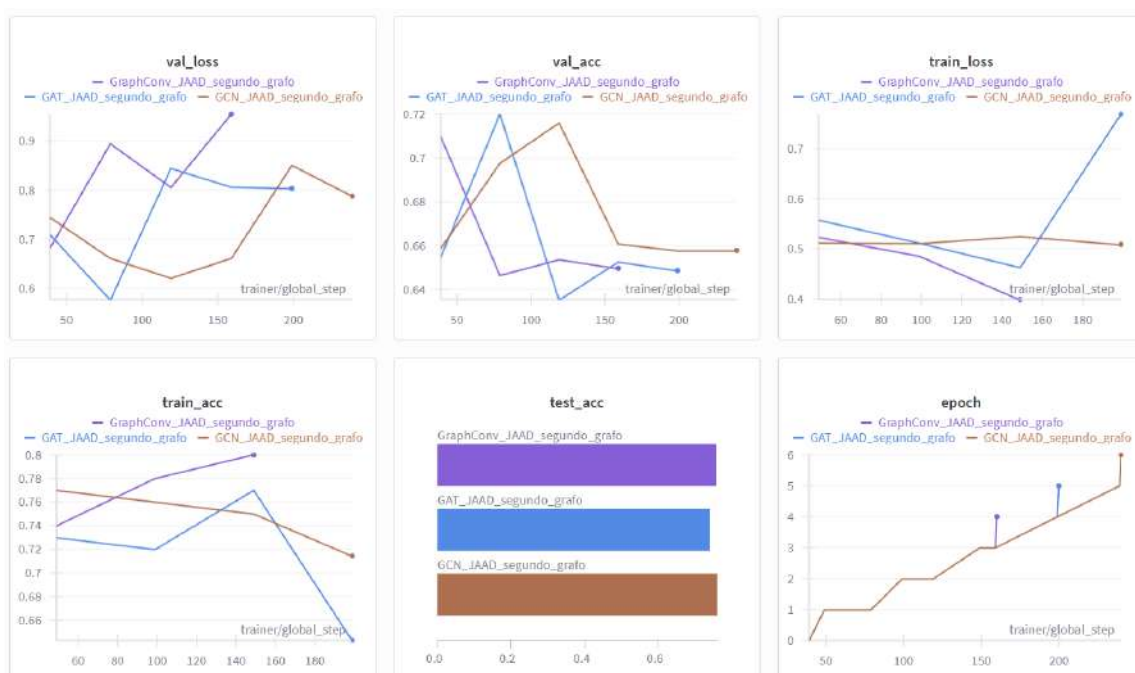


Figura 6.5: Resultados utilizando el segundo grafo

Por último, el tercer grafo (véase Figura 5.1c) combina las aristas de los dos primeros diseños, y muestra los mejores resultados de forma consistente. Al integrar relaciones directas e indirectas, el grafo proporciona una contextualización más rica, permitiendo a los modelos capturar interacciones simultáneas. En este contexto, destacan los valores de exactitud (siendo los más altos). Estos resultados muestran que una estructura de grafo más completa favorece al aprendizaje de patrones complejos (véase Figura 6.6).

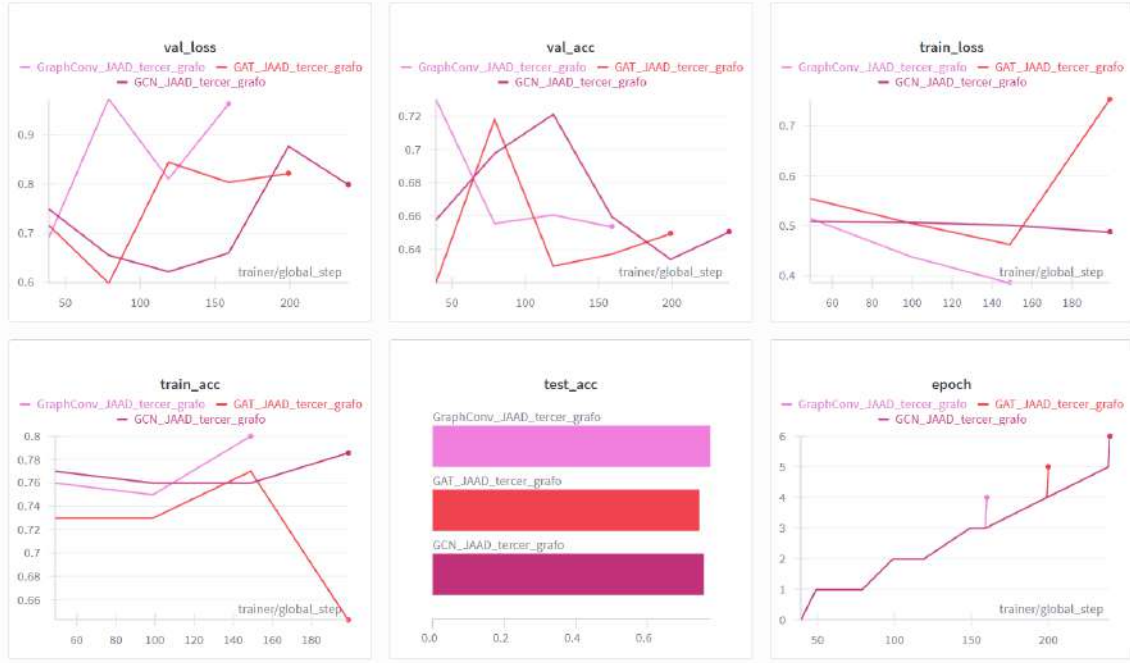


Figura 6.6: Resultados utilizando el tercer grafo

Modelo	Grafo	test_acc
GCN	Primer grafo	0.7590
GCN	Segundo grafo	0.7584
GCN	Tercer grafo	0.7596
GAT	Primer grafo	0.7471
GAT	Segundo grafo	0.7384
GAT	Tercer grafo	0.7473
GraphConv	Primer grafo	0.7731
GraphConv	Segundo grafo	0.7553
GraphConv	Tercer grafo	0.7764

Tabla 6.5: Comparativa de resultados con los tres grafos

6.2.2. Comparativa según el tipo de datos

Una vez analizada la estructura del grafo, es interesante estudiar la influencia del tipo de datos de entrada, comparando tanto la versión numérica del dataset con la versión lingüística. Este análisis permite evaluar si incorporar información semántica aporta mejoras significativas en la predicción de la intención del peatón.

Al utilizar el dataset numérico, los modelos GNN muestran un comportamiento estable y consistente mediante los tres grafos espaciales. Con la arquitectura GCN, los valores de exactitud son muy similares, indicando que la estructura del grafo no tiene un gran peso cuando se utiliza una arquitectura sencilla. En cuanto a GAT, es el que muestra mejores resultados globales en la comparación. Este comportamiento sugiere que los mecanismos de atención aprovechan las conectividades adicionales del grafo, aunque las diferencias siguen sin ser notables. En el caso de GraphConv, los resultados son ligeramente inferiores frente a las otras dos arquitecturas, ya que los modelos más expresivos no siempre garantizan una mejora del rendimiento con información numérica (véase Figura 6.7).

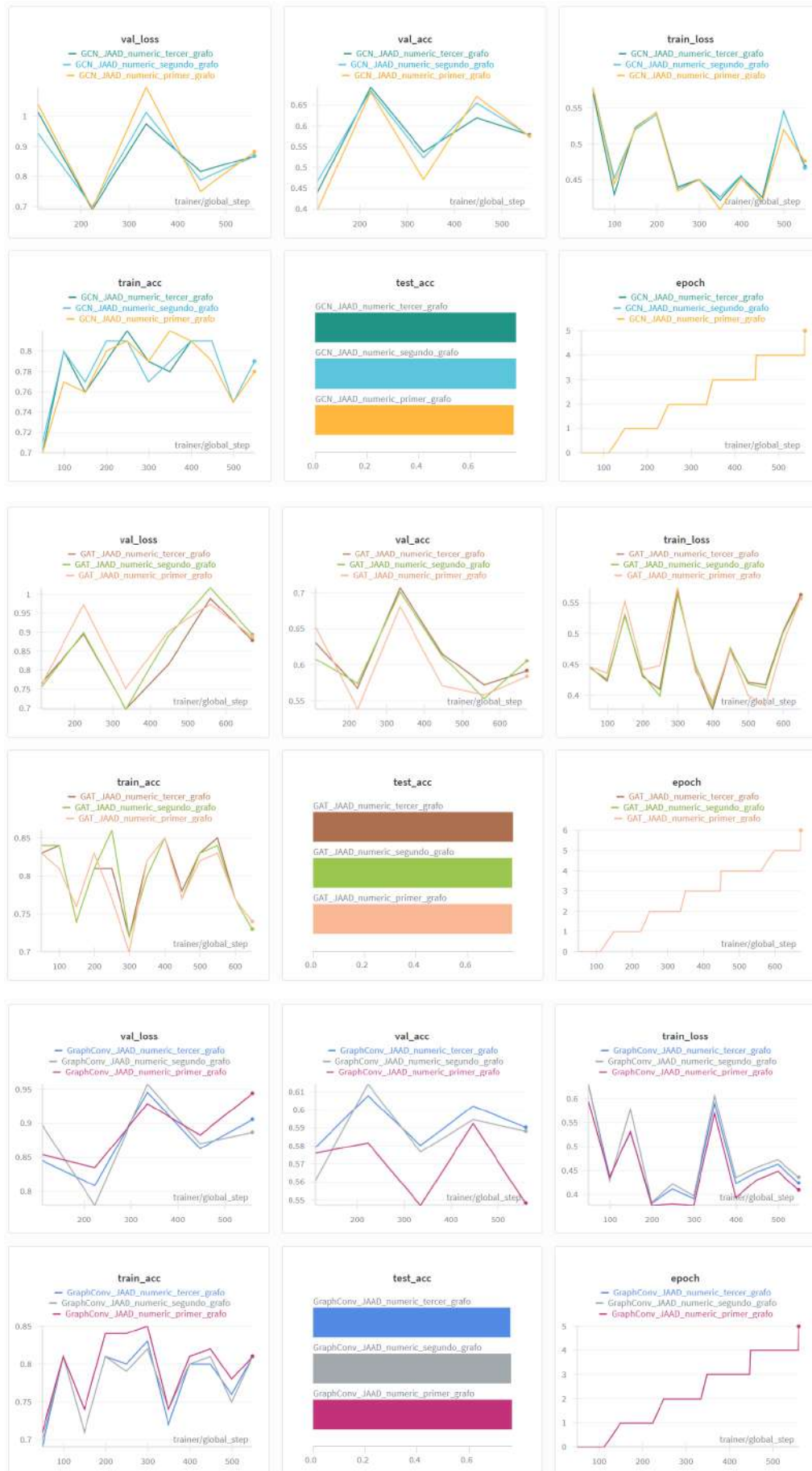


Figura 6.7: Resultados utilizando el dataset numérico para las distintas arquitecturas

Por otra parte, los resultados obtenidos con el dataset lingüístico son muy próximos a los resultados de los datos numéricos. Para GCN, los resultados apenas muestran variación, siendo ligeramente inferiores, por lo que la información lingüística no aporta ventaja con un modelo sencillo. GAT tampoco introduce mejoras, por lo que los mecanismos de atención ya capturan de forma suficiente la información relevante. Por último, el modelo GraphConv es el único que muestra una ligera mejora gracias a la flexibilidad adicional de esta técnica, permitiendo una mejor explotación de la información semántica. Sin embargo, esta ligera mejora sigue siendo moderada y no generalizable a todas las configuraciones (véase Figura 6.8).

Analizando en conjunto ambos datasets (véase Tabla 6.6), los resultados producidos son muy similares, por lo que la representación numérica ya es lo suficiente informativa para el trabajo. La aportación de información lingüística no empeora el rendimiento, pero no es un factor determinante frente al diseño del grafo o el modelo utilizado.

Modelo	Dataset	Grafo	test_acc
GCN	Numérico	Primer grafo	0.7669
GCN	Numérico	Segundo grafo	0.7737
GCN	Numérico	Tercer grafo	0.7735
GCN	Lingüístico	Primer grafo	0.7637
GCN	Lingüístico	Segundo grafo	0.7665
GCN	Lingüístico	Tercer grafo	0.7661
GAT	Numérico	Primer grafo	0.7744
GAT	Numérico	Segundo grafo	0.7760
GAT	Numérico	Tercer grafo	0.7768
GAT	Lingüístico	Primer grafo	0.7709
GAT	Lingüístico	Segundo grafo	0.7688
GAT	Lingüístico	Tercer grafo	0.7752
GraphConv	Numérico	Primer grafo	0.7647
GraphConv	Numérico	Segundo grafo	0.7637
GraphConv	Numérico	Tercer grafo	0.7613
GraphConv	Lingüístico	Primer grafo	0.7709
GraphConv	Lingüístico	Segundo grafo	0.7789
GraphConv	Lingüístico	Tercer grafo	0.7821

Tabla 6.6: Comparativa de resultados con los dos tipos de datasets

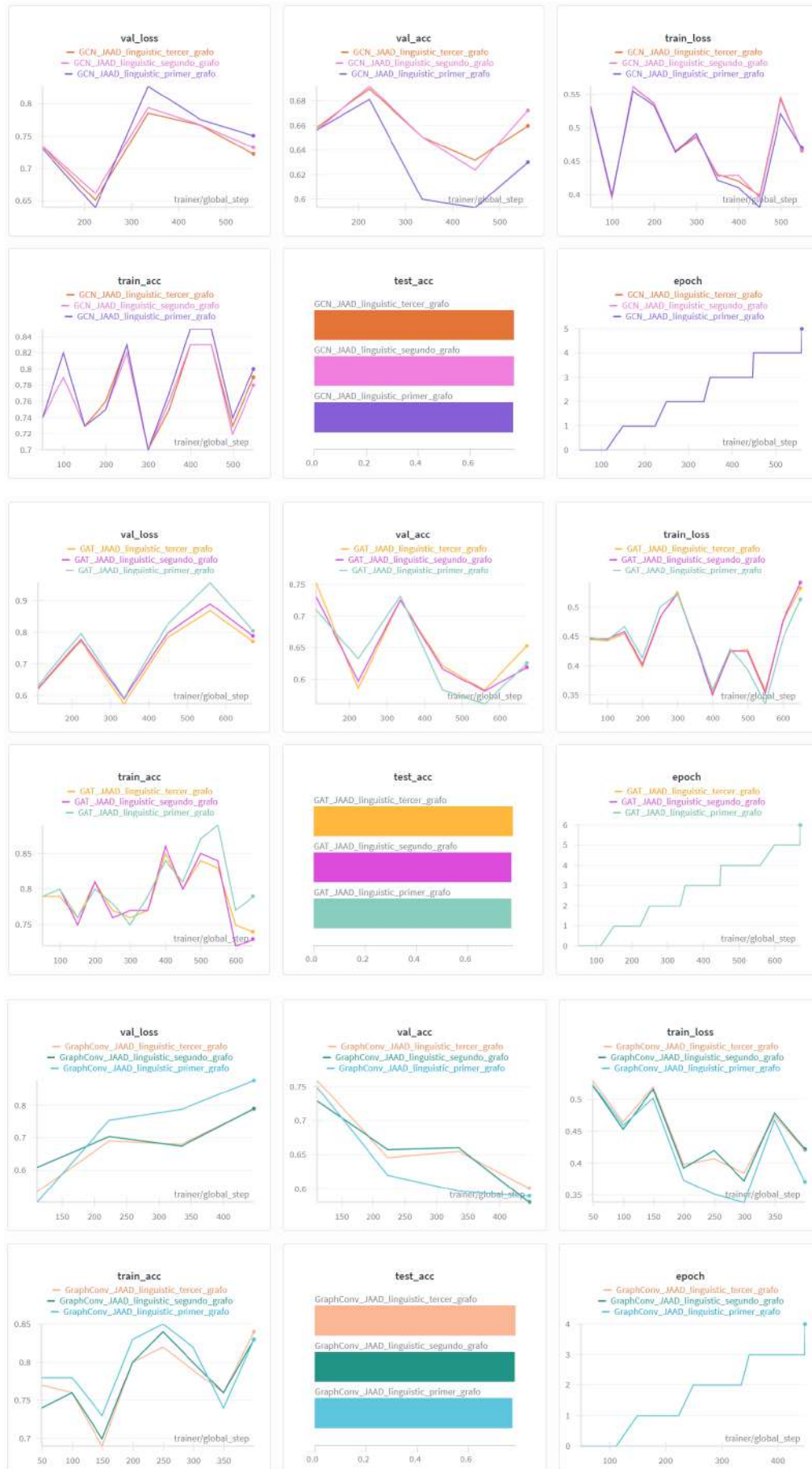


Figura 6.8: Resultados utilizando el dataset lingüístico para las distintas arquitecturas

6.3. Comparativa de arquitecturas GNN

Una vez analizado el diseño del grafo espacial y los datos utilizados, es conveniente comparar las arquitecturas GNN empleadas en los experimentos, evaluando el impacto del modelo sobre el rendimiento del sistema en conjunto con las configuraciones experimentales definidas. En general, los resultados no muestran una arquitectura dominante en todas las configuraciones sobre el resto, pero si se observan tendencias en función de la complejidad del modelo y la capacidad de agregación.

La arquitectura GCN tiene un comportamiento estable y robusto frente a cambios en el diseño y los datos. Sin embargo, en la mayoría de los casos, el valor de la precisión se sitúa ligeramente por debajo de los obtenidos con otras técnicas más expresivas. Esta razón sustenta la idea de que el mecanismo de convolución uniforme limita la capacidad de capturar interacciones de mayor complejidad entre los nodos.

Por otra parte, con GAT se obtienen los resultados más consistentes de los experimentos realizados. Los mecanismos de atención característicos de GAT permiten ponderar de forma desigual el peso de los vecinos, siendo realmente beneficioso en grafos con mayor conectividad. No obstante, las mejoras obtenidas frente a GCN son moderadas, lo que indica que la calidad del grafo tiene más influencia en el mecanismo de atención que el tipo de datos de entrada.

Por último, GraphConv muestra un comportamiento diferente al de los otros dos modelos, ya que algunas configuraciones obtienen resultados inferiores, pero otras obtienen los valores más altos, como por ejemplo con los datos lingüísticos. Este comportamiento se debe a la flexibilidad a la hora de agregar las características, ya que permite explotar mejor las representaciones de los datos, aunque esta ventaja no está siempre presente de forma uniforme.

Este análisis muestra como la arquitectura del modelo influye en el rendimiento, pero su impacto es menor que el del diseño del grafo. Las GCN ofrecen una solución simple pero eficiente, mientras que las GAT y GraphConv capturan patrones de mayor complejidad. Es por esta razón que la elección del modelo ha de ser en conjunto con el diseño del grafo, y no de manera aislada.

6.4. Análisis de robustez y semillas

Una vez realizados los experimentos, es necesario comprobar la robustez del modelo frente a la aleatoriedad inherente en el proceso de entrenamiento. Para ello, se ha realizado un análisis sobre múltiples ejecuciones utilizando distintas semillas aleatorias. Cada modelo ha sido entrenado 30 veces distintas, con una configuración experimental fija, variando únicamente la semilla de inicialización. El objetivo de este análisis es verificar si los resultados obtenidos previamente han sido fruto de la fortuna, o por el contrario, si los modelos convergen a soluciones similares. Este nuevo estudio es de especial interés en un campo como los grafos donde tanto la inicialización como el muestreo tienen influencia sobre el aprendizaje.

En cuanto a la estabilidad general del sistema, podemos observar que los tres modelos presentan una media bastante similar, todas entre 0.76 y 0.77, unas desviaciones típicas bastante bajas, alrededor de 0.008, y unos rangos de variación muy pequeños, entre 0.03 y 0.04. Estos valores indican que los modelos son consistentes, es decir, que los cambios en la semilla no provocan grandes oscilaciones

seed	acc	seed	acc	seed	acc
0	0.7639	10	0.7733	20	0.7540
1	0.7776	11	0.7666	21	0.7517
2	0.7721	12	0.7609	22	0.7841
3	0.7574	13	0.7672	23	0.7593
4	0.7538	14	0.7704	24	0.7778
5	0.7683	15	0.7605	25	0.7620
6	0.7628	16	0.7855	26	0.7696
7	0.7679	17	0.7700	27	0.7609
8	0.7668	18	0.7790	28	0.7675
9	0.7586	19	0.7782	29	0.7670

Tabla 6.7: Comparativa de semillas en GCN

Test Accuracy medio (media)	0.7679
Desviación típica	0.0081
Valor mínimo	0.7539
Valor máximo	0.7855
Rango (máximo - mínimo)	0.0316

Tabla 6.8: Resultados estadísticos con GCN

en el rendimiento, lo cual demuestra que no hay dependencias fuertes de inicialización, el dataset es suficientemente grande y homogéneo para estabilizar el aprendizaje, y el entrenamiento no cae en mínimos muy distintos al variar los parámetros iniciales.

En cuanto a cada modelo, GCN (véase Tabla 6.7) presenta la media más alta de los tres modelos (0.7679), siendo el modelo más estable, ya que tiene menor sensibilidad a la semilla, menor rango y mejor media (véase Tabla 6.8). Mientras que GAT (véase Tabla 6.9) es el modelo con la media más baja (0.7596), la mayor desviación típica (0.0086) y el mayor rango (0.0425), por lo que es menos robusto frente a distintas semillas, ya que la atención hace al modelo más flexible, pero más inestable (véase Tabla 6.10). Por su parte, GraphConv (véase Tabla 6.11) queda en un punto intermedio, ya que es más estable que GAT, pero un poco menos robusto que GCN. El modelo se comporta bien, aunque no logra la consistencia de GCN (véase Tabla 6.12).

Con este análisis se interpreta que:

1. El sistema es robusto, ya que una desviación típica baja indica que no hay “lucky seeds” que inflen el rendimiento artificialmente, por lo que los modelos convergen de forma consistente a soluciones similares. Esta cuestión es muy importante en un problema de grafos, ya que la estructura y la inicialización tienen gran influencia.
2. El test accuracy está dominado por la información del dataset, ya que la variabilidad de semillas es muy pequeña en comparación con la variabilidad entre arquitecturas, tipo de grafo o estrategias temporales, por lo que la limitación está en los datos y no en los limitadores.
3. El modelo GCN parece el mejor candidato final ya que combina la mayor media, la menor varianza, la mejor robustez y la mejor convergencia.

seed	acc	seed	acc	seed	acc
0	0.7651	10	0.7584	20	0.7628
1	0.7702	11	0.7613	21	0.7542
2	0.7643	12	0.7555	22	0.7698
3	0.7603	13	0.7664	23	0.7553
4	0.7641	14	0.7532	24	0.7576
5	0.7569	15	0.7580	25	0.7540
6	0.7670	16	0.7374	26	0.7323
7	0.7561	17	0.7632	27	0.7712
8	0.7643	18	0.7748	28	0.7567
9	0.7548	19	0.7620	29	0.7578

Tabla 6.9: Comparativa de semillas en GAT

Test Accuracy medio (media)	0.7596
Desviación típica	0.0086
Valor mínimo	0.7323
Valor máximo	0.7748
Rango (máximo - mínimo)	0.0425

Tabla 6.10: Resultados estadísticos con GAT

seed	acc	seed	acc	seed	acc
0	0.7534	10	0.7546	20	0.7571
1	0.7694	11	0.7546	21	0.7517
2	0.7738	12	0.7698	22	0.7675
3	0.7597	13	0.7817	23	0.7555
4	0.7641	14	0.7616	24	0.7685
5	0.7698	15	0.7696	25	0.7715
6	0.7647	16	0.7635	26	0.7626
7	0.7672	17	0.7597	27	0.7510
8	0.7458	18	0.7567	28	0.7630
9	0.7765	19	0.7645	29	0.7662

Tabla 6.11: Comparativa de semillas en GraphConv

Test Accuracy medio (media)	0.7632
Desviación típica	0.0080
Valor mínimo	0.7459
Valor máximo	0.7817
Rango (máximo - mínimo)	0.0358

Tabla 6.12: Resultados estadísticos con GraphConv

6.5. Modelado avanzado

El objetivo del modelado avanzado es extender la base ya creada incorporando para ello tanto información temporal como una estructura adicional, para poder analizar si estas dependencias pueden mejorar la predicción. Para ello, se van a estudiar diversas estrategias temporales sobre el grafo, además de una extensión sobre la estructura del peatón. Estos avances son aplicados sobre el *pipeline* ya validado, permitiendo realizar un análisis comparativo incremental y controlado.

6.5.1. Modelado temporal

El comportamiento del peatón es esencialmente un fenómeno dinámico, ya que la intención del cruce no depende únicamente del estado actual, sino también de la propia evolución a lo largo del tiempo. Esta razón ha motivado la explotación de nuevas estrategias para incorporar dependencias temporales en la representación del grafo, con el objetivo de capturar tanto el movimiento continuo como el contexto temporal inmediato del peatón. Para ello, se han planteado tres posibles escenarios. Inicialmente, un escenario básico donde solo se conecte temporalmente el nodo peatón. A continuación, un escenario avanzado donde se conecten temporalmente todos los nodos. Y por último, la implementación de las *sliding windows*.

6.5.1.1. Conexión temporal básica

La aproximación base consiste en introducir conexiones temporales entre los nodos correspondientes al peatón en frames consecutivos (véase Figura 5.2). En este diseño, cada nodo representado por un peatón en un instante temporal se conecta con el nodo representado por el mismo peatón en el siguiente frame, manteniendo de esta forma las relaciones espaciales ya comentadas dentro de cada instante. Con esta estructura se busca que el modelo aprenda transiciones temporales básicas sin incrementar de manera excesiva la complejidad del grafo.

Modelo	Grafo	test_acc
GCN	Primer grafo	0.7803
GCN	Segundo grafo	0.7761
GCN	Tercer grafo	0.7766
GAT	Primer grafo	0.7814
GAT	Segundo grafo	0.7718
GAT	Tercer grafo	0.7772
GraphConv	Primer grafo	0.7777
GraphConv	Segundo grafo	0.7843
GraphConv	Tercer grafo	0.7806

Tabla 6.13: Comparativa de resultados con el grafo espaciotemporal básico

Los resultados obtenidos (véase Figura 6.9). muestran una clara mejora de la consistencia en el rendimiento respecto a los grafos espaciales. La precisión incrementa ligeramente en la mayoría de configuraciones (véase Tabla 6.13). Esta mejoría muestra que los modelos son capaces de aprovechar la información temporal local, aprendiendo patrones del peatón respecto al cruce. Además, también

ha mejorado la estabilidad durante el entrenamiento, por lo que las nuevas conexiones han servido como regularizador implícito.

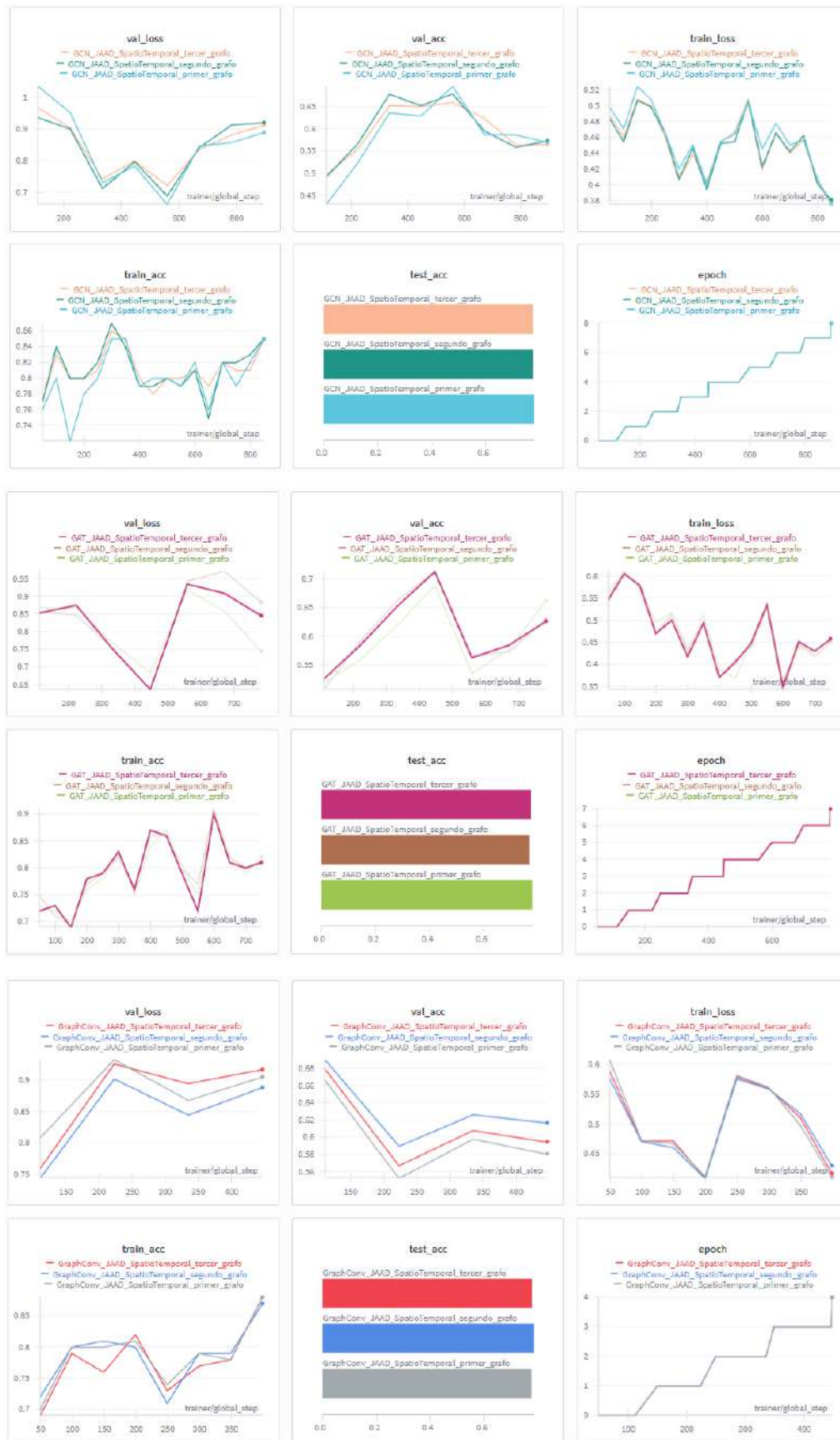


Figura 6.9: Resultados utilizando el grafo espaciotemporal básico

6.5.1.2. Conexión temporal completa

En cuanto al escenario avanzado, se basa en una ampliación del primero, donde todos los nodos equivalentes son conectados entre frames consecutivos (véase Figura 5.3). Este nuevo enfoque genera un grafo espaciotemporal mucho más denso, ya que todas las variables evolucionan conjuntamente en el tiempo. A pesar del aumento de la complejidad, el rendimiento medio se mantiene prácticamente constante, por lo que los modelos son capaces de manejar este incremento de complejidad sin afectar a la capacidad predictiva.

Modelo	Grafo	test_acc
GCN	Primer grafo	0.7683
GCN	Segundo grafo	0.7771
GCN	Tercer grafo	0.7778
GAT	Primer grafo	0.7765
GAT	Segundo grafo	0.7786
GAT	Tercer grafo	0.7781
GraphConv	Primer grafo	0.7825
GraphConv	Segundo grafo	0.7833
GraphConv	Tercer grafo	0.7805

Tabla 6.14: Comparativa de resultados con el grafo espaciotemporal completo

Las mejoras obtenidas en cuanto a resultado (véase Tabla 6.14) frente a la versión base son limitadas (véase Figura 6.10). Esto sugiere que gran parte de la información temporal relevante ya se captura con la conexión del peatón entre frames. Algunas variables aportan cambios muy suaves en instantes consecutivos, lo que puede aportar información redundante cuando explícitamente se conectan estos nodos.

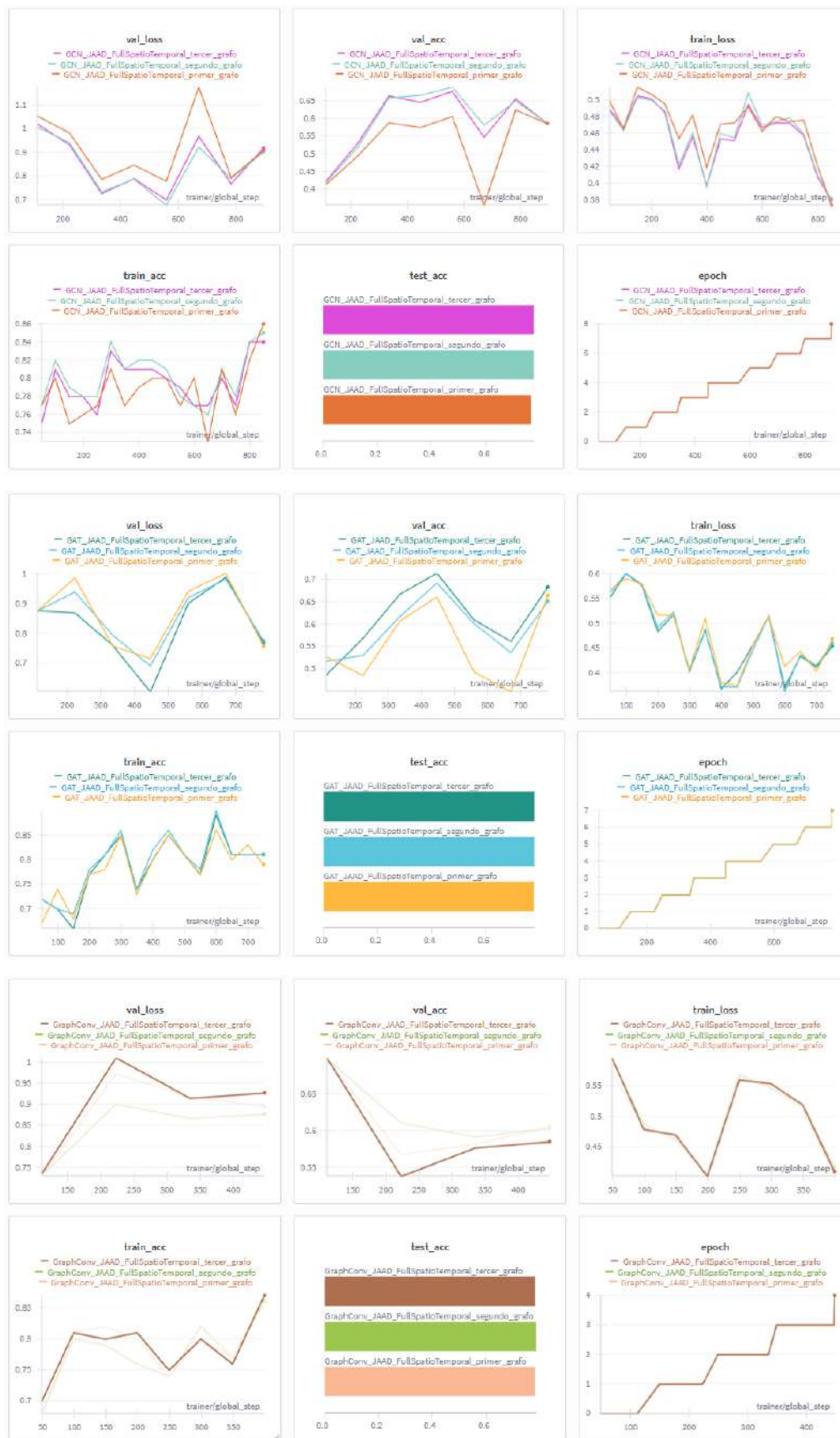


Figura 6.10: Resultados utilizando el grafo espaciotemporal completo

6.5.1.3. Ventanas deslizantes

Por último, se ha desarrollado un modelado alternativo basado en ventanas temporales deslizantes (*sliding windows*) (véase Figura 5.4). En este caso, se agrupan varios frames consecutivos, generando una representación agregada gracias a un promedio temporal de las características, construyendo así un único grafo representativo de la ventana. Con este método se busca capturar la evolución temporal local sin tener que conectar explícitamente ningún nodo.

Modelo	Grafo	test_acc
GCN	Primer grafo	0.7791
GCN	Segundo grafo	0.7795
GCN	Tercer grafo	0.7723
GAT	Primer grafo	0.7765
GAT	Segundo grafo	0.7763
GAT	Tercer grafo	0.7749
GraphConv	Primer grafo	0.7856
GraphConv	Segundo grafo	0.7891
GraphConv	Tercer grafo	0.7832

Tabla 6.15: Comparativa de resultados con las ventanas deslizantes

Los resultados obtenidos (véase Tabla 6.15) (véase Figura 6.11). utilizando las *sliding windows* son muy competitivos, superando en ocasiones al grafo espaciotemporal completo. El hecho de incorporar información temporal local ayuda al modelo a captar tendencias dinámicas a corto plazo, sin introducir complejidad ni ruido que pueda tener el grafo espaciotemporal completo.

En cuanto a las arquitecturas utilizadas, GCN mantiene un comportamiento estable, con ligeras mejoras, GAT presenta también una mejora general en el rendimiento, y GraphConv destaca como el modelo que más aprovecha las relaciones temporales, mostrando los mejores resultados.

Realizando un análisis en conjunto, estos experimentos han demostrado que incorporar información temporal mejora el rendimiento y la estabilidad del sistema. Estos resultados sugieren que, para un dataset como JAAD, el contexto temporal local es más relevante que la conectividad temporal global.



Figura 6.11: Resultados utilizando las ventanas deslizantes

6.5.2. Modelado multipeatón

En los enfoques que han sido planteados a lo largo del trabajo, cada instancia del dataset era modelada de manera independiente a nivel de peatón, sin contemplar la posibilidad de que hubiera otros usuarios presentes en la misma escena. No obstante, en situaciones reales de tráfico, la intención de un peatón de cruzar o no la calzada no depende únicamente de su propio estado, sino que también esta decisión puede verse influida por el comportamiento, posición o intención de otros peatones cercanos. Esta limitación ha sido la que ha motivado la introducción de un nuevo modelado basado en una representación multipeatón, donde se considera de forma explícita la coexistencia de más de un peatón en una escena.

El objetivo principal de esta parte del estudio es enriquecer la representación del contexto, permitiendo así que el modelo capture las relaciones sociales implícitas y patrones colectivos que no pueden ser representados cuando se estudia a un peatón de forma aislada.

6.5.2.1. Experimento multipeatón básico

En este modo, cada escena es definida como una combinación única de video y frame, donde se representa como un grafo independiente. En este grafo, cada nodo corresponde a un peatón de la escena, las características del nodo se basan en las mismas variables utilizadas en los enfoques anteriores (attention, orientation, proximity, distance, action y zebra_cross) y las aristas representan las relaciones entre peatones dentro de la escena. El grafo está completamente conectado, ya que cada peatón está conectado con el resto de peatones que se encuentran en la escena, así el modelo es capaz de aprender que relaciones son relevantes y cuáles no. Las escenas que contienen únicamente a un peatón se descartan para este modo, ya que no aportan información adicional en comparación con los enfoques anteriores, y no permiten modelar las nuevas interacciones.

Como el grafo representa una escena completa y no un peatón individual, es necesario definir una etiqueta única por grafo. Para ello, se han definido dos estrategias posibles:

1. **Etiqueta por mayoría:** La etiqueta en la escena se asigna en función de la clase más frecuente entre los peatones que se encuentren, capturando así el comportamiento dominante en la escena, mostrando un accuracy entre 0.67 y 0.695.
2. **Etiqueta positiva si al menos cruza un peatón:** En este caso, la escena se considera positiva si existe al menos un peatón que esté cruzando, independientemente del comportamiento del resto de peatones de la escena, logrando un accuracy aproximado de 0.635 (véase Tabla 6.16) (véase Figura 6.12).

La etiqueta por mayoría es la que ha mostrado mejores resultados, ya que se trata de una aproximación más conservadora y menos sensible a casos extremos. Aun así, es interesante como ambas representaciones permiten analizar cómo la definición del objetivo puede afectar al rendimiento del modelo.

Con este enfoque lo que se logra es que la decisión final no dependa exclusivamente de un peatón, sino del estado conjunto de la escena, capturando así patrones colectivos y posibles dependencias implícitas entre peatones.

Modelo	Tipo de etiqueta	test_acc
GCN	Mayoría	0.6956
GCN	Un peatón cruzando	0.6356
GAT	Mayoría	0.6956
GAT	Un peatón cruzando	0.6356
GraphConv	Mayoría	0.6713
GraphConv	Un peatón cruzando	0.6356

Tabla 6.16: Comparativa de resultados con el modelo multipeatón básico



Figura 6.12: Resultados utilizando el modelo multipeatón básico

Este nuevo modelado puede interpretarse como una generalización del enfoque de un solo peatón. Mientras que en el modelo original se aprendía a predecir la intención de cruce basándose únicamente en las características individuales del usuario (y, en ocasiones, dependencias temporales), el nuevo enfoque introduce de manera explícita el contexto social de cada muestra. Este nuevo cambio no añade nueva información, sino que reorganiza la información ya existente de una forma más coherente con

un problema real, ayudando a evaluar hasta qué punto las interacciones entre peatones contribuyen a mejorar la capacidad predictiva del modelo.

Aunque sí que es cierto que el enfoque multipeatón enriquece la nueva representación del contexto, sigue estando limitado debido a las características disponibles del dataset, ya que las interacciones aprendidas están condicionadas a variables observables y no capturan algunos factores externos como pueden ser las señales del entorno, la intención latente o la comunicación implícita entre peatones. No obstante, esta nueva formulación constituye un gran avance entre el modelado individual y arquitecturas más complejas.

6.5.2.2. Experimento multipeatón extendido

El modelado anterior permite capturar las interacciones sociales dentro de una escena concreta, siendo una representación estática y básica, limitada a un único instante temporal. Sin embargo, en escenarios de tráfico reales, la intención de cruce del usuario no es una decisión instantánea, sino un proceso dinámico que evoluciona a lo largo del tiempo, estando condicionado por el propio contexto reciente de la escena y la evolución inmediata de ésta. Con el nuevo objetivo de superar esta limitación, se ha desarrollado una nueva implementación del enfoque multipeatón que incorpora explícitamente la dependencia temporal entre escenas. Con este nuevo planteamiento se pretende modelar ya no solo las interacciones sociales dentro de una escena concreta, sino también la continuidad temporal del comportamiento de un colectivo.

En este nuevo modo, cada muestra se construye a partir de una ventana temporal de tres escenas consecutivas ($t-1$, t , $t+1$), donde cada escena representa una combinación única de video y frame. El grafo resultante combina estas tres escenas en una estructura espaciotemporal, donde cada peatón se representa como un nodo independiente, aunque corresponda al mismo individuo en otro instante temporal. En cada escena, los nodos se conectan mediante aristas espaciales completamente conectadas, modelando así las interacciones sociales locales, y entre escenas consecutivas, se añaden aristas temporales para conectar al mismo peatón en distintos frames, preservando así la coherencia temporal. De este modo, el grafo final contiene información social (dentro de cada escena) y temporal (conectando escenas), permitiendo al modelo aprender patrones de evolución del comportamiento colectivo. Las ventanas temporales que contienen únicamente a un peatón son descartadas, ya que no aportan información adicional a este análisis y rompen con la consistencia del aspecto social.

Las características de los nodos se construyen de la misma manera, empleando los mismos features (attention, orientation, proximity, distance, action y zebra_cross), garantizando así la compatibilidad de las comparaciones entre experimentos. Cada peatón mantiene su representación individual, pero ahora se ve enriquecida por la información procedente tanto de otros peatones como del propio comportamiento en instantes adyacentes. De esta forma, se evita introducir nueva información externa y se centra en reorganizar la información ya existente de una forma más alienada con el problema real, reforzando así la coherencia semántica del grafo.

En cuanto al etiquetado, como el grafo representa una ventana temporal completa, la etiqueta se asigna utilizando la escena central (t) de la ventana. Esta decisión permite que la predicción esté condicionada tanto por el contexto anterior como por el posterior, pero que el objetivo del modelo

siga siendo la estimación de la intención de cruce en un instante concreto. La etiqueta se define utilizando las dos mismas técnica que en el enfoque multipeatón básico.

Modelo	Tipo de etiqueta	test_acc
GCN	Mayoría	0.6164
GCN	Un peatón cruzando	0.6387
GAT	Mayoría	0.6555
GAT	Un peatón cruzando	0.6266
GraphConv	Mayoría	0.5913
GraphConv	Un peatón cruzando	0.5991

Tabla 6.17: Comparativa de resultados con el modelo multipeatón extendido



Figura 6.13: Resultados utilizando el modelo multipeatón extendido

En este caso, el segundo criterio obtiene mejores resultados (véase Tabla 6.17) (véase Figura 6.13), lo que sugiere que la incorporación del contexto temporal modifica la información disponible para el modelo y la forma en que explota esta información, evidenciando que la efectividad del criterio de eti-

quetado depende directamente del tipo de representación del grafo, ya que en el enfoque multipeatón básico, el primer criterio obtenía mejores resultados.

Con este nuevo enfoque, la decisión final de cruce ya no depende del estado instantáneo de un peatón o de una escena aislada, sino que ahora depende del comportamiento conjunto de la escena y su evolución temporal. El modelo ahora es capaz de capturar tanto patrones colectivos entre peatones como señales temporales relevantes para la estimación.

Este enfoque puede interpretarse como una generalización natural del modelado básico del concepto multipeatón, ya que, mientras el modelo anterior capturaba exclusivamente las interacciones sociales instantáneas, el nuevo concepto introduce continuidad temporal explícita, permitiendo analizar como dichas interacciones evolucionan y se refuerzan a lo largo del tiempo.

Esta estructura puede entenderse como un primer paso hacia un grafo de conocimiento espacio-temporal, donde cada escena actúa como una unidad semántica conectada temporalmente con otras escenas. Con este planteamiento se definen las bases para arquitecturas más expresivas, capaces de integrar distinta información contextual, social y temporal de forma conjunta.

Capítulo 7

Discusión

En este capítulo se va a realizar una interpretación de los resultados obtenidos, además de hacer un estudio sobre los datos utilizados en los experimentos.

7.1. Interpretación de resultados

Los resultados obtenidos mediante los distintos experimentos permiten sacar interpretaciones relevantes sobre el impacto de la representación del grafo, el modelo GNN utilizado, el dataset empleado, además de la incorporación de información temporal. Todos estos experimentos evidencian que el rendimiento del sistema está más condicionado por la calidad y coherencia del modelado que por mejoras o incrementos en la complejidad del grafo o modelo.

Los experimentos iniciales con grafos espaciales (véase Sección 6.2.1) han demostrado que la estimación de la intención del peatón puede abordarse de manera efectiva con estructuras simples, siempre que estas estén alineadas de forma semántica con el problema. Las diferencias entre grafos espaciales muestran que no todas las relaciones estructurales son útiles por igual. En concreto, las relaciones entre peatón y entorno inmediato ofrecen resultados más estables que otras con conexiones adicionales.

Por otro lado, las comparativas entre arquitecturas GNN (véase Sección 6.3) muestran que tanto GCN como GAT y GraphConv presentan comportamientos similares, con precisiones medias muy parecidas, con diferencias moderadas. No obstante, en líneas generales GraphConv se beneficia de representaciones enriquecidas, sobretodo con dependencias temporales, por lo que tiene mayor capacidad para explotar información estructural con relaciones de mayor complejidad.

En cuanto al análisis entre datos numéricos y lingüísticos (véase Sección 6.2.2) indica que ambos tipos de datos contienen información redundante para la predicción. Las precisiones obtenidas son muy similares en ambos casos, lo que sugiere que la transformación lingüística no introduce nueva discriminación, sino que reformula las variables originales. Esto es relevante ya que el rendimiento del modelo depende más de las propias variables que del formato en el que se representen.

Incorporar información temporal ha supuesto un punto de inflexión en el desarrollo de los experimentos (véase Sección 6.5.1). Tanto ambos grafos espaciotemporales como las ventanas deslizantes

mejoran la estabilidad del entrenamiento y, en muchos casos, el rendimiento respecto a los grafos espaciales. Sin embargo, estos resultados muestran que el contexto temporal local es suficiente para capturar la mayor parte de la información relevante. Como el grafo espaciotemporal extendido no supera consistentemente al básico, conectar todas las variables puede introducir redundancia o ruido, sin aportar beneficios claros. En esta línea, las *sliding windows* son una solución eficaz, ya que capturan tendencias temporales sin incrementar la complejidad del grafo.

El modelado multipeatón (véase Sección 6.5.2) ha resultado una nueva dimensión para la interpretación de resultados, al centrar la atención en la escena en vez de en el peatón individual. En el experimento multipeatón básico (véase Sección 6.5.2.1), los resultados muestran que se permite capturar patrones colectivos, pero con una pérdida notable de la precisión. Este descenso es esperable, ya que la nueva formulación puede resultar más ambigua y dependiente del etiquetado. Por otro lado, el experimento multipeatón extendido (véase Sección 6.5.2.2) refuerza el concepto multipeatón al incorporar ventanas temporales entre escenas. Esta extensión no se traduce en una mejora del rendimiento, ya que, en líneas generales, la precisión disminuye, lo que sugiere que combinar dependencias sociales y temporales dificulta al aprendizaje cuando los datos son limitados, o las señales disponibles no son lo suficiente expresivas. No obstante, estos nuevos enfoques añaden un gran valor conceptual, ya que sienta las bases para modelos espaciotemporales más ricos.

Todos estos resultados muestran un equilibrio entre expresividad y complejidad. Este análisis demuestra que, en contextos reales como el que se representa en JAAD, una representación diseñada con cuidado puede ser igual de determinante que la arquitectura GNN utilizada, además de que la incorporación de información tanto temporal como social debe añadirse de forma controlada.

7.2. Limitaciones del dataset

A pesar de los resultados obtenidos mediante los experimentos, resulta necesario identificar posibles limitaciones asociadas al dataset utilizado. Estas limitaciones no invalidan los resultados, pero delimitan su interpretación, siendo una posible línea de mejora en el futuro.

7.2.1. Revisión de muestras del dataset

Una gran limitación viene de la propia naturaleza de JAAD, debido al posible ruido intrínseco en las muestras. Para realizar un análisis de ruido y ambigüedad en el dataset, es conveniente revisar algunas muestras de forma manual para ver si a simple vista se puede detectar algunas muestras que carezcan de sentido, por lo que opaquen así el correcto funcionamiento de los modelos.

Existen diversas combinaciones que pueden generar ruido en el modelo, como, por ejemplo:

1. **Un peatón caminando hacia la calzada, pero etiquetado como *not-crossing*:** Esta clase de frames pueden confundir al modelo, ya que la acción instantánea (en este caso *walking*) parece que pueda indicar cruce, pero la etiqueta aún es negativa. El cambio a *crossing* se produce de golpe, apenas habiendo diferencia con la muestra inmediatamente anterior.

Muestra 16 (véase Figura 7.1): video_0044,44,0_44_202b,not-looking,166.4,1,1,7.49,walking,not-crossing



Figura 7.1: Muestra 16

Muestra 17 (véase Figura 7.2): video_0044,45,0_44_202b,not-looking,159.95,0,1,7.4,walking,crossing



Figura 7.2: Muestra 17

2. **Un peatón se encuentra de pie, pero etiquetado como *crossing*:** Este tipo de frames son una fuente habitual de ruido, ya que son frames justo antes o después del cruce, donde el peatón se encuentra parado.

Muestra 147 (véase Figura 7.3): video_0057,39,0_57_257b,not-looking,85.26,0,1,9.01,standing,crossing



Figura 7.3: Muestra 147

3. **Largas series del mismo peatón con comportamiento repetitivo y etiqueta fija:** Esto introduce una alta correlación temporal, por lo que puede que los modelos GNN acaben memorizando secuencias.

Muestra 407 (véase Figura 7.4): video_0078,65,0_78_416b,not-looking,348.7,0,0,4.48,walking,crossing

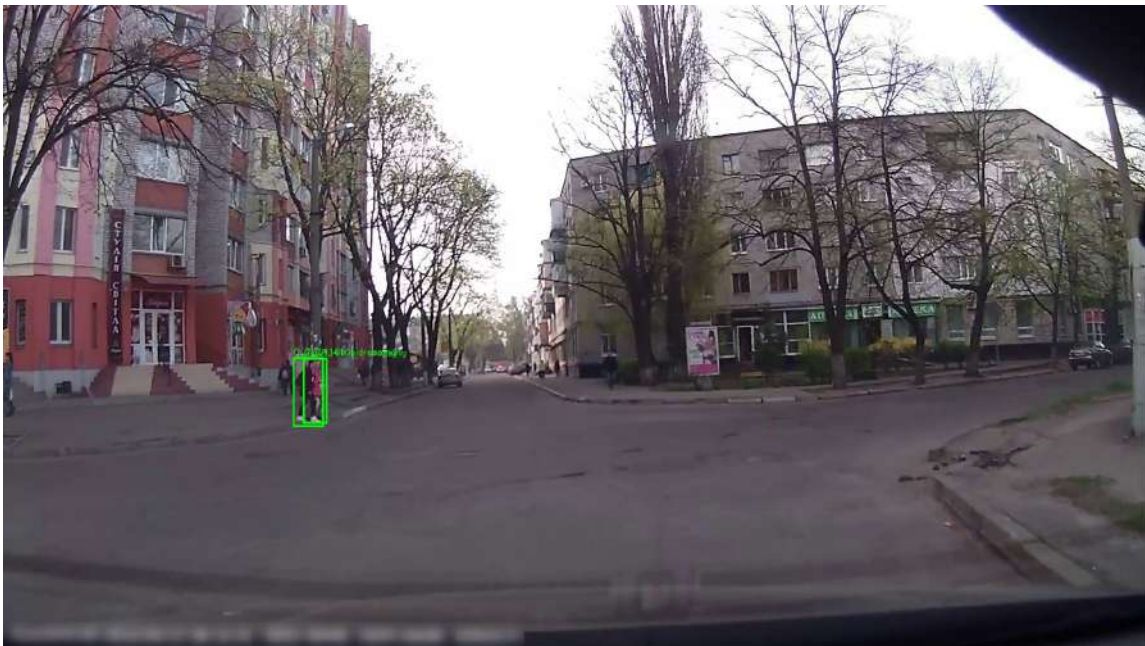


Figura 7.4: Muestra 407

Muestra 527 (véase Figura 7.5): video_0078,125,0_78_416b,not-looking,312.86,0,0,3.18,walking,crossing

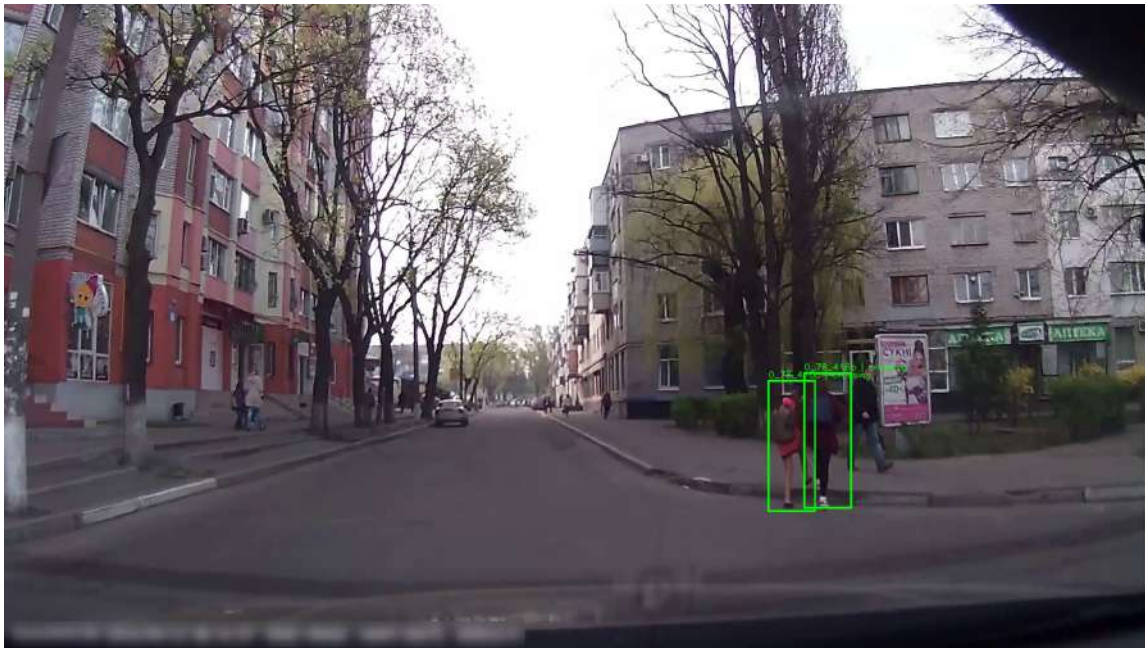


Figura 7.5: Muestra 527

Si analizamos todo en conjunto, podemos observar que algunas muestras generan ruido y cierta ambigüedad en el dataset. Entre los casos ambiguos en el dataset, podemos destacar los siguientes:

1. **Peatón caminando, pero con etiqueta *not-crossing*:** Hay 4491 muestras donde el peatón se encuentra caminando (*walking*) pero su etiqueta es de no cruzar. Esto ocurre cuando el peatón camina por la acera o se mueve sin intención de cruzar, por lo que se produce una ambigüedad real. Esto implicará que el modelo tenga dificultades para aprender cuando caminar implica una intención real de cruce.
2. **Peatón quieto, pero con etiqueta *crossing*:** Hay 756 muestras donde el peatón se encuentra quieto (*standing*) pero su etiqueta es de cruzar. Esto suele ocurrir en instantes iniciales del cruce donde el peatón está mirando, frames justo antes de cruzar o etiquetas retrasadas/adelantadas respecto al movimiento real. Esto ocasiona que el modelo pueda aprender erróneamente que estar quieto también predice un cruce, por lo que disminuye la precisión.
3. **Orientación extremadamente alta:** En el dataset, la categoría *orientation* no es categórica, sino que es una categoría numérica entre 0 y 360. Esto puede crear ruido ya que 0° y 360° son equivalentes, y se pueden producir saltos bruscos debido al *wrap-around*, es decir, el sistema se envuelve al superar el límite. Si el peatón está en 359° y cambia su orientación ligeramente, en vez de pasar a 361° , pasaría a 1° , por lo que se realiza un cambio en la orientación. Puede tener impacto al introducir ruido angular y hacer que los modelos confundan cambios reales con reglas numéricas.

Muestra 3188 (véase Figura 7.6): video_0215,204,0_215_1572b,not-looking,359.17,0,1,1.58,walking,not-crossing

Muestra 3190 (véase Figura 7.7): video_0215,205,0_215_1572b,not-looking,2.26,0,1,1.44,walking,not-crossing

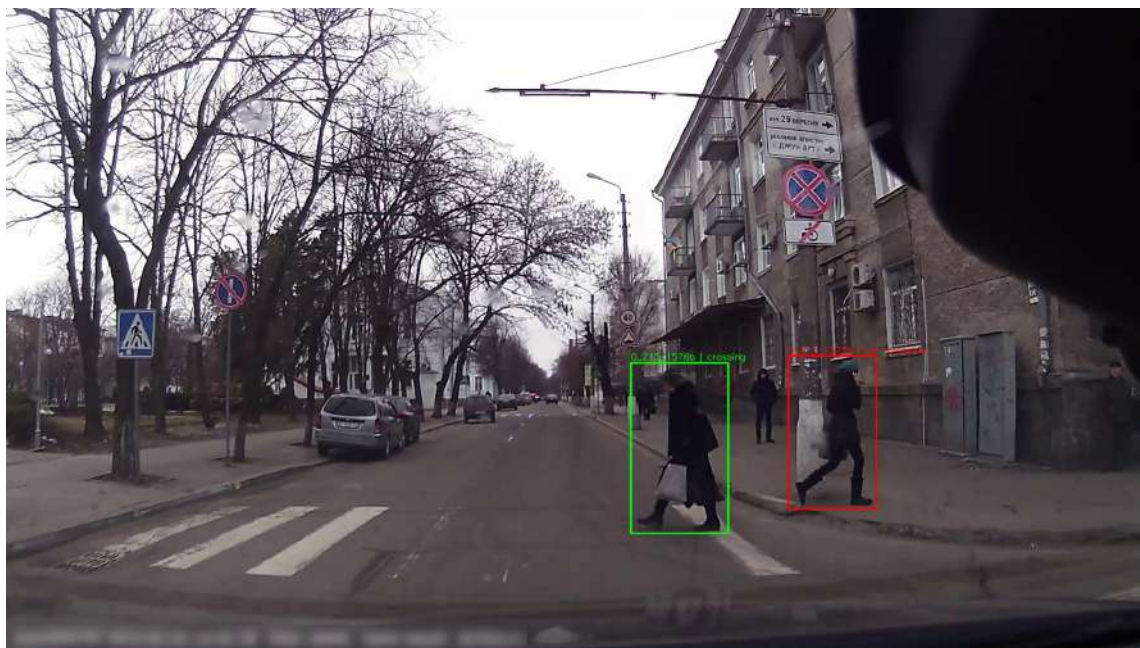


Figura 7.6: Muestra 3188

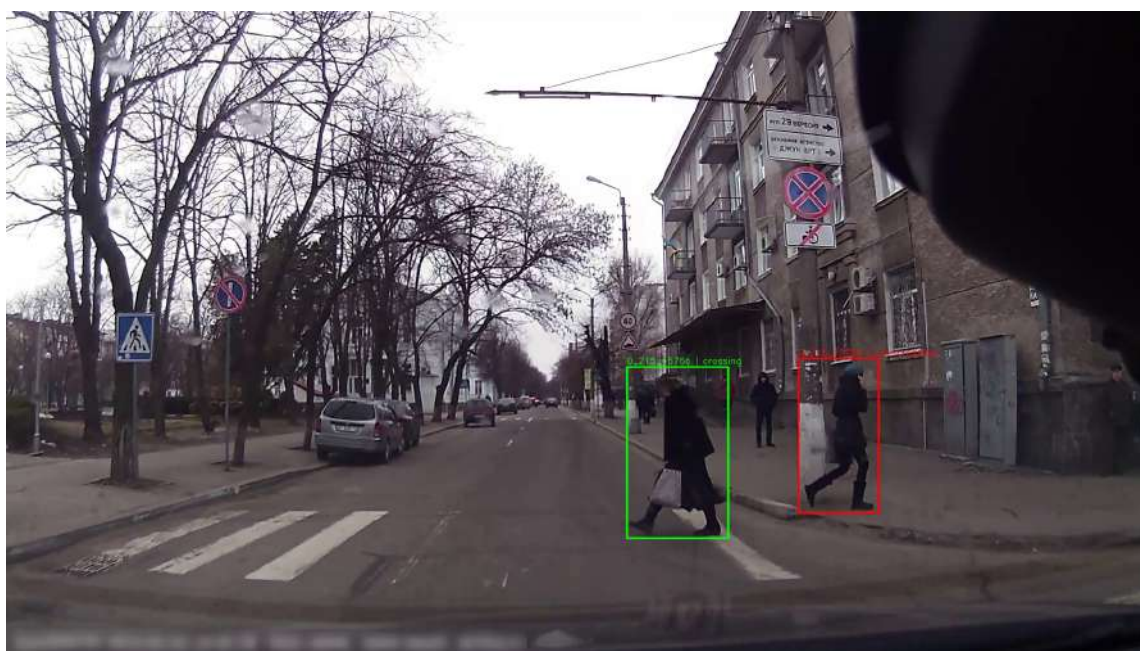


Figura 7.7: Muestra 3190

Estos casos descritos de ruido y ambigüedad se relacionan con inconsistencias entre las columnas, por lo que se pueden destacar las siguientes inconsistencias:

1. **Etiqueta de acción (*action*) y etiqueta de cruce (*cross*):** Previamente, ya hemos visto que hay 4491 muestras donde el peatón está caminando, pero sin cruzar, y 756 muestras donde el peatón está parado, pero cruzando. Si sumamos todas estas muestras, obtenemos 5247 filas, lo cual representa el 37% del dataset que contiene 14000 muestras, un dato bastante elevado. Esto se debe a que JAAD contiene muchos frames que no representan correctamente la acción actual, sino la intención global del usuario.

2. **Etiqueta de orientación (*orientation*) demasiado variable:** En los peatones reales, hay cierto ruido angular debido a errores humanos del anotador, *jitter* (ruido no deseado) propio del detector y cambios bruscos entre frames que no representan con precisión la orientación real del peatón.

Todo este análisis confirma que el dataset JAAD contiene un ruido natural que es típico de datasets sobre el comportamiento humano, además de algunas ambigüedades que afectan a los modelos de precisión sobre la intención del peatón. Este ruido explica por qué la etiqueta de cruce (*cross*) no siempre refleja la acción instantánea.

7.2.2. Análisis del dataset

Una vez abordado todo el modelado mediante GNN, es muy interesante observar que por muchas mejoras que se añadan, el resultado no llega a mejorar significativamente, por lo que cabe preguntarse si el problema no está en los modelos empleados sino en los datos utilizados. Para ello, se va a realizar un análisis para explorar la calidad de los datos para poder evaluar la capacidad predictiva de las características disponibles, la existencia de ruido y la posibilidad de aplicar técnicas de *pruning*. Para ello, se ha diseñado un *workflow* en KNIME que permite hacer una evaluación rápida, reproducible y visual del comportamiento de los datos sin tener que introducir sesgos derivados del uso de modelos profundos.

El flujo del análisis parte del dataset original (tanto train como test) y construye una megavariable textual donde se concatenan todas las características relevantes de cada muestra(en este caso son *attention*, *orientation*, *proximity*, *distance* y *action*) (véase Figura 7.8). Esta representación permite tratar cada instancia como una descripción estructurada sobre el comportamiento del usuario, manteniendo toda la información completa, pero en un único campo. El objetivo del estudio no es entrenar un modelo final en particular, sino evaluar si la información disponible en el dataset es suficientemente discriminativa de manera conjunta.

Una vez construida esta megavariable, el *workflow* se centra en detectar descripciones idénticas de un comportamiento que aparecen con etiquetas distintas de cruce. Este análisis es clave ya que este escenario indica ambigüedad intrínseca en los datos y casos de ruido semántico, por lo que se medirá la conflictividad de cada combinación única de features. Este tipo de ambigüedad no está relacionada con errores de medición, sino con la propia naturaleza del ser humano observado en las escenas, ya que realmente una combinación idéntica de características puede llevar a distintas decisiones debido al contexto no capturado por el dataset. Esto introduce un límite estructural en la capacidad predictiva, independientemente del modelo entrenado sobre los datos.

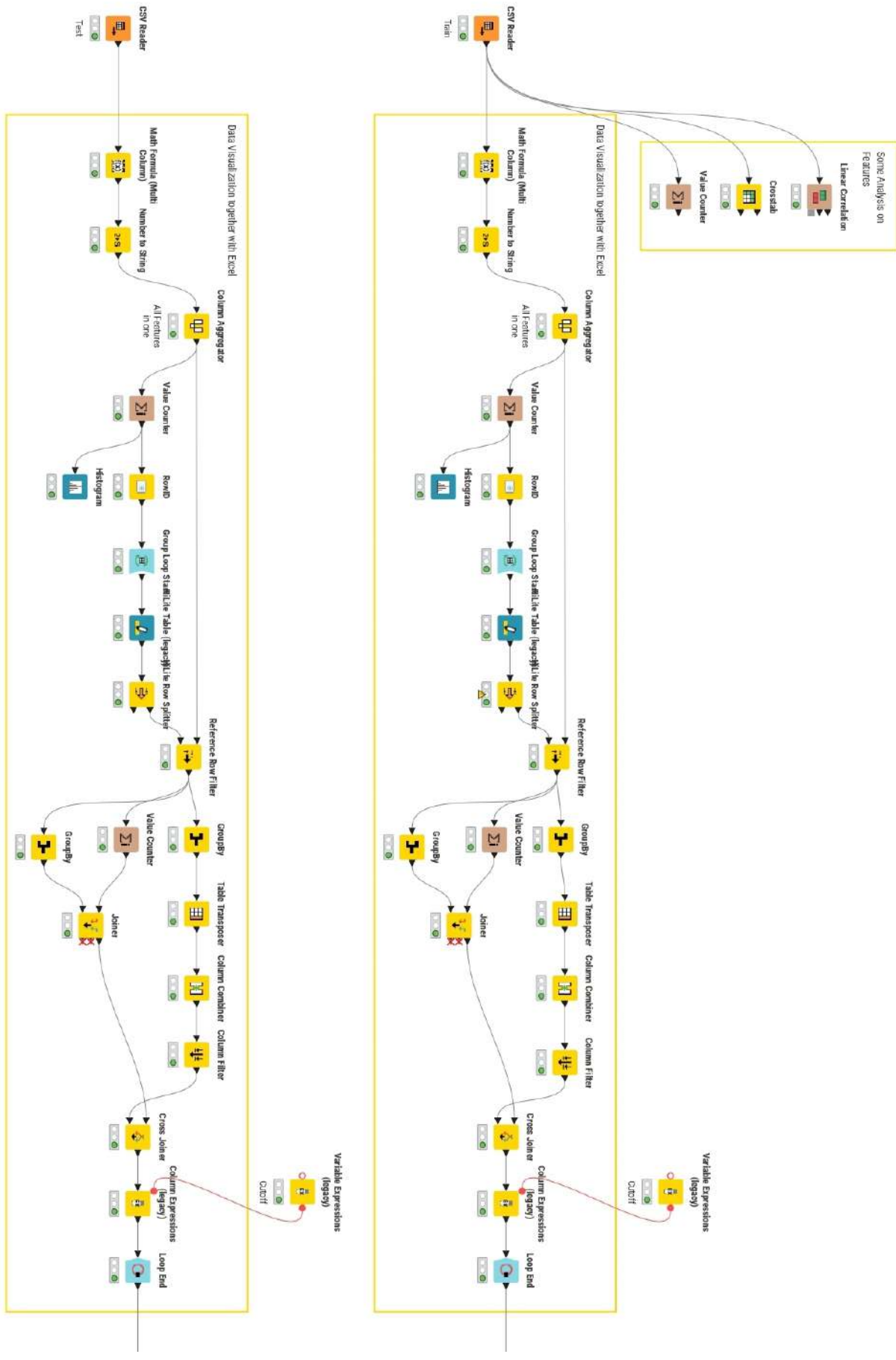


Figura 7.8: Primera parte del flujo del análisis en KNIME

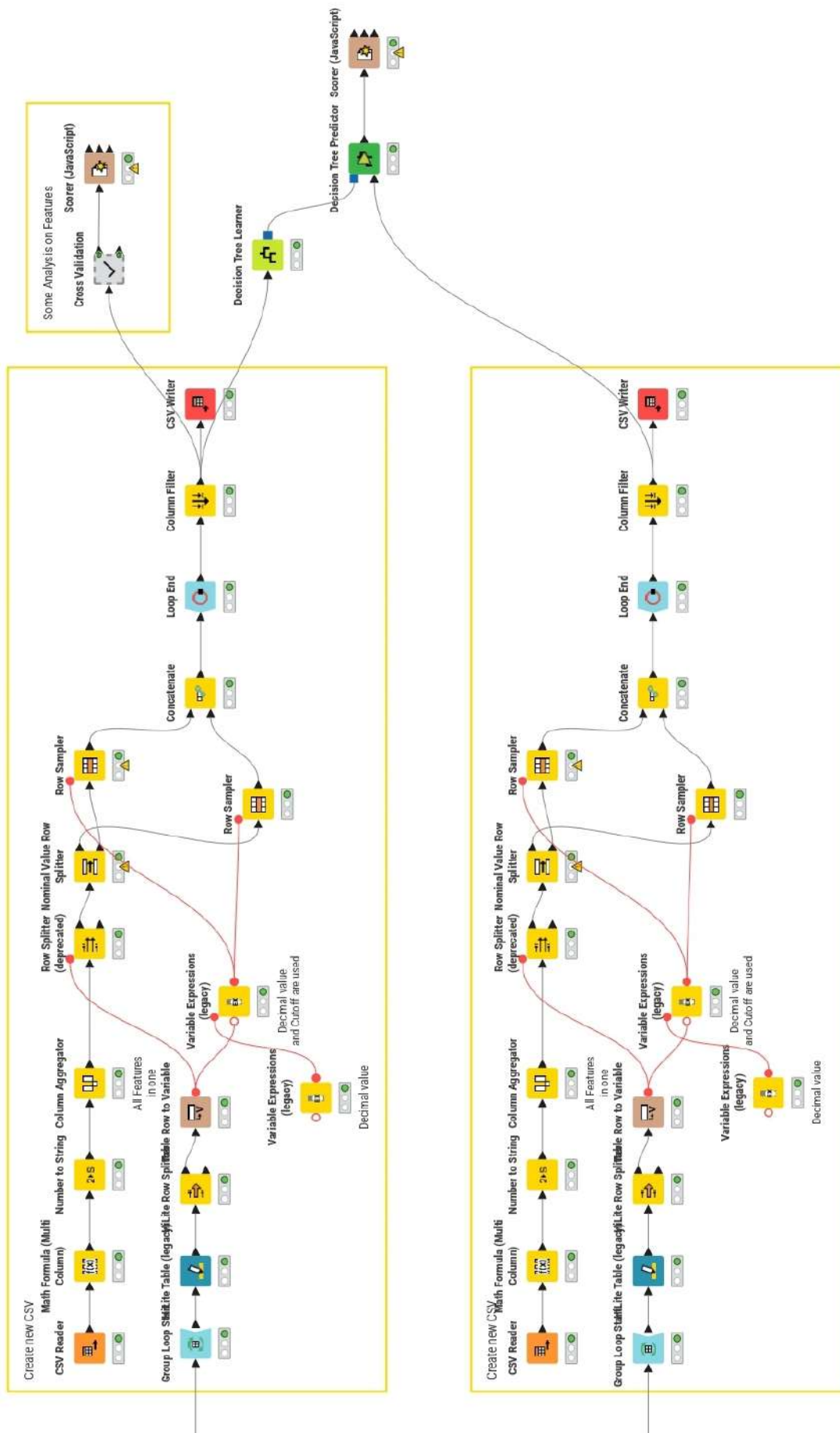


Figura 7.9: Segunda parte del flujo del análisis en KNIME

Para comprobar el grado de conflicto, se va a calcular un ratio definido mediante la siguiente fórmula:

$$\text{grado de conflicto} = \frac{\text{mín}(\text{crossing}, \text{not-crossing})}{\text{máx}(\text{crossing}, \text{not-crossing})}$$

Figura 7.10: Ecuación del grado de conflicto

Las propiedades de la ecuación del grado de conflicto son las siguientes:

- Si el resultado obtenido es cercano a 0, la combinación está claramente dominada por una clase.
- Si el resultado obtenido es cercano a 1, la combinación es altamente ambigua.

Una vez obtenido este grado, es necesario decidir si hay que hacer *pruning* mediante el uso de un *cutoff*. Para ello, se establece un umbral configurable en este *cutoff*. Este umbral es utilizado para marcar combinaciones problemáticas, ayudando así a decidir si es necesario eliminar o mantener ciertas combinaciones. Si el valor obtenido en la ecuación es mayor que el umbral, la combinación obtenida es conflictiva, pero si el valor obtenido en la ecuación es menor que el umbral, la combinación obtenida es aceptable. Esto permite obtener un criterio objetivo y razonable para poder evaluar la necesidad de depuración del dataset y analizar el impacto potencial del ruido sin la necesidad de entrenar modelos complejos.

Una vez realizado este proceso, se realiza una validación cruzada con árboles de decisión que muestra un accuracy de 0.82, mientras que la evaluación utilizando un *learner* y *predictor* logra un accuracy de 0.638 (véase Figura 7.9). El valor de *cross validation* representa una estimación optimista pero estable del rendimiento medio esperado del modelo con el dataset, siendo una buena medida de capacidad predictiva promedio. Por otro lado, el valor del *learner* y *predictor* refleja un escenario más exigente y realista, siendo especialmente sensible al ruido y a combinaciones conflictivas. Este comportamiento es relevante en datasets donde existen combinaciones de entrada que no permiten una separación entre clases, ya que las métricas obtenidas durante la validación cruzada tienden a sobrestimar la verdadera capacidad de generalización del modelo.

La clave radica en que existen combinaciones idénticas de features asociadas tanto a *crossing* como *not-crossing*, algunas de ellas teniendo ratios cercanos a 1, lo que implica un límite teórico del rendimiento. En la validación cruzada, este conflicto se reparte, por lo que el modelo aprende ambas posibilidades, inflando así el resultado obtenido en el accuracy.

Si comparamos el escenario descrito previamente donde se construye toda la megavariante y un escenario simple donde únicamente se aplica *Decision Tree* (véase cualquier modelo) (véase Figura 7.11), podemos ver que este último logra un accuracy mejor que el flujo más grande. Una vez hecho este flujo, toca preguntarse a qué se debe esto.

Si vemos la evaluación sobre la megavariante utilizando los árboles de decisión, se aprecia que el resultado es de 0.638, mientras que en el flujo más sencillo es de 0.683, y esto se puede deber a diversos motivos:

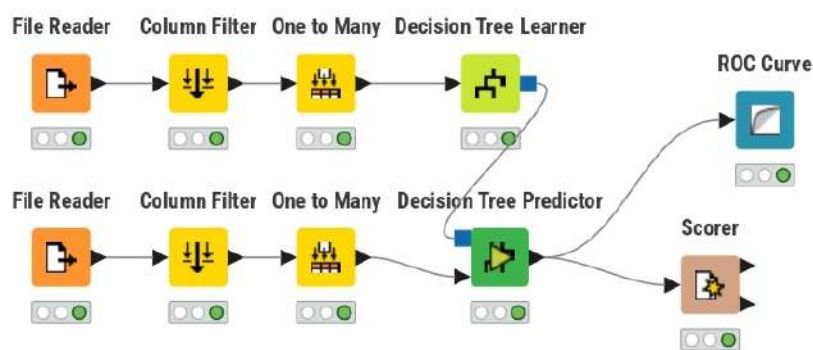


Figura 7.11: Flujo del análisis utilizando *Decision Tree* en KNIME

- El *workflow* de calidad penaliza explícitamente al dataset: La creación de una megavariable puede empeorar deliberadamente la calidad del dataset, pero mejora la fiabilidad del análisis, ya que detecta combinaciones conflictivas, aplicando *pruning* cuando estas filas sean ambiguas, por lo que el accuracy es más bajo pero también más "honesto".
- El árbol directo se beneficia del ruido: Al entrenar al árbol de decisión directamente, el modelo ve todas las features por separado, pudiendo sobreajustar algunas combinaciones. Algunas reglas no generalizan, pero ayudan en ese test, lo que produce un ligero aumento artificial del accuracy.

El valor obtenido utilizando el flujo más sencillo (0.683) es bastante interesante, ya que, a partir de los mismos datos utilizados en las GNN, el accuracy obtenido es menor que la media obtenida de todas las pruebas realizadas anteriormente (0.76-0.78). Con esto vemos que los modelos avanzados responden mejor frente a estos datos que algunos modelos más simples como lo puede ser *decision tree*, ya que los modelos GNN aprovechan mejor las relaciones estructurales, las dependencias temporales y las relaciones semánticas entre features. Este análisis sugiere que el dataset presenta cierta ambigüedad que limita el máximo alcanzable mediante modelos basados en las características disponibles del dataset. Por lo tanto, las mejoras adicionales que se hagan en el modelado solo lograrán ligeros incrementos, ya que no se introduce información contextual adicional.

Capítulo 8

Conclusiones y futuros trabajos

En este último capítulo, se exponen tanto las conclusiones del trabajo como líneas futuras de investigación.

8.1. Conclusiones

A modo de resumen, en el trabajo se han realizado las siguientes aportaciones:

- Inicialmente, se ha diseñado y validado un *pipeline* completo para poder validar la intención de cruce del peatón mediante GNN, utilizando desde grafos espaciales sencillos hasta representaciones temporales más complejas.
- En cuanto a las arquitecturas, se ha realizado una comparación entre GNN, en concreto GCN, GAT y GraphConv, mostrando que todas presentan rendimientos similares en grafos espaciales, pero que GraphConv destaca con dependencias temporales y estructurales más avanzadas.
- Se ha estudiado el impacto al usar distintos tipos de datos, comparando datos numéricos con lingüísticos, concluyendo que ambos contienen información equivalente, por lo que el rendimiento está más condicionado por la propia información disponible que por su formato.
- Se ha introducido información temporal utilizando grafos espaciotemporales y *sliding windows*, con una mejora en la estabilidad y rendimiento del sistema gracias al contexto temporal local. No obstante, una conectividad temporal excesiva puede introducir redundancia.
- Además, se ha propuesto un nuevo enfoque con respecto al resto del desarrollo, el concepto multipeatón, donde se representan interacciones sociales dentro de una escena, además de una extensión espaciotemporal de este enfoque, sirviendo de base para modelos colectivos.
- Para analizar la robustez, se ha hecho un análisis utilizando semillas aleatorias, confirmando que los modelos convergen de forma estable, por lo que los resultados no dependen de inicializaciones arbitrarias.

- Por último, se ha desarrollado un análisis sobre los datos utilizados, identificando posible ruido, ambigüedad semántica o límites estructurales, además de un estudio independiente sobre la calidad del dataset mediante KNIME.

En conclusión, todos estos resultados muestran que, un problema realista pero complejo como la estimación de intención de un usuario puede ser representado gracias a un grafo y su semántica. La implementación de las Graph Neural Networks (GNN) ha permitido explotar distintos tipos de relaciones, ya sean espaciales, temporales y sociales. Por otra parte, el rendimiento alcanzable no solo depende del modelo empleado, sino también de la calidad de los datos utilizados.

8.2. Futuros trabajos

A partir de los resultados obtenidos y las limitaciones identificadas, existen diversas líneas de trabajo futuro para poder ampliar y mejorar el enfoque planteado:

- Resultaría interesante enriquecer el dataset utilizado con nuevas variables de contexto, como por ejemplo, información de las señales del entorno, la velocidad de los vehículos o la distribución de la vía. Con nueva información contextual, se podría reducir ambigüedad en los datos, mejorando la capacidad predictiva.
- Además, el modelado multipeatón podría mejorarse con arquitecturas más expresivas, como pueden ser los *graph transformers* o modelos híbridos de GNN que capturen interacciones sociales más complejas a largo plazo. También cabe mencionar la distancia entre los peatones, ya que ahora mismo se analiza el escenario sin saber realmente a qué distancia está cada peatón del resto.

Finalmente, cabe destacar que el desarrollo presentado no se limita exclusivamente al uso del dataset JAAD, sino que la arquitectura ha sido diseñada con un propósito modular y generalizable. Esto permite que el *pipeline* pueda ser reutilizado en otros contextos similares.

Un ejemplo claro es el trabajo que va a desarrollar un compañero del grado donde abordará un problema similar utilizando cámaras de tráfico globales y grafos. Este trabajo se titula “Estimación de intenciones de cruce mediante modelos basados en grafos y datos de cámaras de tráfico globales” y tiene como objetivo el estudio de un sistema predictivo capaz de estimar la intención de cruce de peatones utilizando como fuente de datos cámaras de tráfico instaladas en diferentes entornos viales del mundo.

El diseño de este trabajo puede servir como base directa para la construcción de su nuevo sistema, adaptando la fuente de datos y las variables observadas en los vídeos. Esto refuerza el valor del trabajo debido a la reutilización dentro de una línea de investigación amplia como es el estudio de la intención de los usuarios, en este caso, en la vía.

Bibliografía

- [1] “Real academia española.” [Online]. Disponible en: <https://dle.rae.es/autonom%C3%ADa>
- [2] A. N. Melo, L. F. Herrera-Quintero, C. Salinas, y M. A. Sotelo, “Knowledge-based explainable pedestrian behavior predictor,” in *2024 IEEE Intelligent Vehicles Symposium (IV)*. IEEE, 2024, pp. 3348–3355.
- [3] Z. Wu, S. Pan, F. Chen, G. Long, C. Zhang, y P. S. Yu, “A comprehensive survey on graph neural networks,” *IEEE transactions on neural networks and learning systems*, vol. 32, no. 1, pp. 4–24, 2020.
- [4] S. A. Bouhsain, S. Saadatnejad, y A. Alahi, “Pedestrian intention prediction: A multi-task perspective,” *arXiv preprint arXiv:2010.10270*, 2020.
- [5] A. Rasouli, I. Kotseruba, T. Kunic, y J. K. Tsotsos, “Pie: A large-scale dataset and models for pedestrian intention estimation and trajectory prediction,” in *Proceedings of the IEEE/CVF international conference on computer vision*, 2019, pp. 6262–6271.
- [6] J. Lian, Y. Luo, X. Wang, L. Li, G. Guo, W. Ren, y T. Zhang, “Dual-stgat: Dual spatio-temporal graph attention networks with feature fusion for pedestrian crossing intention prediction,” *IEEE Transactions on Intelligent Transportation Systems*, 2025.
- [7] B. Yu, H. Yin, y Z. Zhu, “Spatio-temporal graph convolutional networks: A deep learning framework for traffic forecasting,” *arXiv preprint arXiv:1709.04875*, 2017.
- [8] V. Kosaraju, A. Sadeghian, R. Martín-Martín, I. Reid, H. Rezatofighi, y S. Savarese, “Social-bigat: Multimodal trajectory forecasting using bicycle-gan and graph attention networks,” *Advances in neural information processing systems*, vol. 32, 2019.
- [9] H. Alghodhaifi y S. Lakshmanan, “Holistic spatio-temporal graph attention for trajectory prediction in vehicle–pedestrian interactions,” *Sensors*, vol. 23, no. 17, p. 7361, 2023.
- [10] J. McCarthy *et al.*, “What is artificial intelligence,” 2007.
- [11] S. Russell, P. Norvig, y A. Intelligence, “A modern approach,” *Artificial Intelligence. Prentice-Hall, Egnlewood Cliffs*, vol. 25, no. 27, pp. 79–80, 1995.
- [12] M. Learning, “Tom mitchell,” *Publisher: McGraw Hill*, p. 31, 1997.

- [13] S. Singh, R. Kumar, S. Payra, y S. K. Singh, “Artificial intelligence and machine learning in pharmacological research: bridging the gap between data and drug discovery,” *Cureus*, vol. 15, no. 8, 2023.
- [14] W. L. Hamilton, *Graph representation learning*. Morgan & Claypool Publishers, 2020.
- [15] Y. LeCun, Y. Bengio, y G. Hinton, “Deep learning,” *nature*, vol. 521, no. 7553, pp. 436–444, 2015.
- [16] I. Goodfellow, Y. Bengio, A. Courville, y Y. Bengio, *Deep learning*. MIT press Cambridge, 2016, vol. 1, no. 2.
- [17] D. J. Matich, “Redes neuronales: Conceptos básicos y aplicaciones,” *Universidad Tecnológica Nacional, México*, vol. 41, pp. 12–16, 2001.
- [18] A. Longa, G. Pellegrini, y G. Santin, “PytorchGeometricTutorial,” Oct. 2021. [Online]. Disponible en: <https://github.com/AntonioLonga/PytorchGeometricTutorial>
- [19] J. Gilmer, S. S. Schoenholz, P. F. Riley, O. Vinyals, y G. E. Dahl, “Neural message passing for quantum chemistry,” in *International conference on machine learning*. Pmlr, 2017, pp. 1263–1272.
- [20] W. L. Hamilton, R. Ying, y J. Leskovec, “Representation learning on graphs: Methods and applications,” *arXiv preprint arXiv:1709.05584*, 2017.
- [21] P. Lippe, “UvA Deep Learning Tutorials: Tutorial 7 — Graph Neural Networks,” https://uvadlc-notebooks.readthedocs.io/en/latest/tutorial_notebooks/tutorial7/GNN_overview.html, 2022.
- [22] H. Cai, V. W. Zheng, y K. C.-C. Chang, “A comprehensive survey of graph embedding: Problems, techniques, and applications,” *IEEE transactions on knowledge and data engineering*, vol. 30, no. 9, pp. 1616–1637, 2018.
- [23] B. Perozzi, R. Al-Rfou, y S. Skiena, “Deepwalk: Online learning of social representations,” in *Proceedings of the 20th ACM SIGKDD international conference on Knowledge discovery and data mining*, 2014, pp. 701–710.
- [24] A. Grover y J. Leskovec, “node2vec: Scalable feature learning for networks,” in *Proceedings of the 22nd ACM SIGKDD international conference on Knowledge discovery and data mining*, 2016, pp. 855–864.
- [25] S. M. Kazemi, R. Goel, K. Jain, I. Kobyzev, A. Sethi, P. Forsyth, y P. Poupart, “Representation learning for dynamic graphs: A survey,” *Journal of Machine Learning Research*, vol. 21, no. 70, pp. 1–73, 2020.
- [26] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimeshein, L. Antiga *et al.*, “Pytorch: An imperative style, high-performance deep learning library,” *Advances in neural information processing systems*, vol. 32, 2019.

- [27] P. Geometric, “Pytorch geometric documentation,” <https://pytorch-geometric.readthedocs.io/en/latest/>, 2026, accessed: 2026-01-27.
- [28] Weights y Biases, “Weights and biases documentation,” 2023, accessed: 2026-01-27. [Online]. Disponible en: <https://docs.wandb.ai/>
- [29] L. Pathak, K. Desai, C. Kela, y T. Patel, “Molecular classification with graph convolutional networks: Exploring the mutag dataset for mutagenicity prediction,” *International Journal of Innovative Science and Research Technology*, vol. 9, no. 8, pp. 2545–2550, 2024. [Online]. Disponible en: <https://doi.org/10.38124/ijisrt/IJISRT24AUG1084>
- [30] A. Rasouli, I. Kotseruba, y J. K. Tsotsos, “Are they going to cross? a benchmark dataset and baseline for pedestrian crosswalk behavior,” in *ICCVW*, 2017, pp. 206–213.
- [31] —, “JAAD: Joint attention in autonomous driving dataset,” https://data.nvision2.eecs.yorku.ca/JAAD_dataset/, 2017, accessed: 2026-02-07.
- [32] M. R. Berthold, N. Cebron, F. Dill, T. R. Gabriel, T. Kötter, T. Meinel, P. Ohl, K. Thiel, y B. Wiswedel, “Knime-the konstanz information miner: version 2.0 and beyond,” *AcM SIGKDD explorations Newsletter*, vol. 11, no. 1, pp. 26–31, 2009.
- [33] T. Kipf, “Semi-supervised classification with graph convolutional networks,” *arXiv preprint arXiv:1609.02907*, 2016.
- [34] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Lio, y Y. Bengio, “Graph attention networks,” *arXiv preprint arXiv:1710.10903*, 2017.
- [35] C. Morris, M. Ritzert, M. Fey, W. L. Hamilton, J. E. Lenssen, G. Rattan, y M. Grohe, “Weisfeiler and leman go neural: Higher-order graph neural networks,” in *Proceedings of the AAAI conference on artificial intelligence*, vol. 33, no. 01, 2019, pp. 4602–4609.
- [36] A. Rodríguez Hurtado, “Estimación de la intención de los usuarios de la vía mediante redes neuronales basadas en grafos,” <https://github.com/adri-rh/tfg>, 2026, repositorio GitHub, consultado el 8 de febrero de 2026.

2025-2026